

Bachelor's Thesis
(Course Code: X_400628)

Benchmarking Data Architecture Patterns for Precision Agriculture IoT Telemetry

by

Alessandro Coatto
(Student Number: 2770462)

Submitted in partial fulfillment of the requirements
for the degree of
Bachelor of Science
in
Computer Science
at the
Vrije Universiteit Amsterdam

August 24, 2026

First Supervisor
Jeewanie Jayasinghe Arachchillage
VU Amsterdam

Daily Supervisor
Qiuke Li
Agurotech

Second Reader
Shahab Ud Din
VU Amsterdam

Benchmarking Data Architecture Patterns for Precision Agriculture IoT Telemetry

Alessandro Coatto
Vrije Universiteit Amsterdam
Amsterdam, NL
a.m.coatto@student.vu.nl

ABSTRACT

Precision agriculture platforms ingest a continuous, append-only stream of soil telemetry and serve it back as dashboard aggregations over windows spanning days to a full growing season. Command Query Responsibility Segregation (CQRS) and the Lambda Architecture are routinely recommended for such read/write asymmetry, but their benefit for agricultural IoT workloads has not been measured against a well-optimised single-store baseline. This thesis investigates such a benchmark. Three prototypes — a single-store MongoDB baseline, a CQRS variant materialising a read model from a Kafka event stream, and a Lambda Architecture variant combining periodic batch recomputation with a Redis speed layer — were implemented over a common ingest path, storage engine, and synthetic dataset, and measured at 1,000, 10,000, and 50,000 sensors across five query types. The baseline serves point lookups and short range scans in under 3 ms at every scale, and sustains the offered write rate throughout. Its cost is confined to read-time aggregation: the full-season summary rises from a mean of 48 to 82 ms across that range, with a 95th percentile of 256 ms. Both separated architectures hold that query below 5 ms at every scale, a separation of roughly 25×, and differ by a fraction of a millisecond. Operational complexity nevertheless orders them strictly, by component count and by the number of write paths requiring consistency, though not by code volume. For this workload class CQRS obtains the full read-side benefit at lower operational complexity, while the Lambda Architecture's additional layer yields no measurable aggregation advantage.

1 INTRODUCTION

Modern agriculture is undergoing a data-driven transformation. The proliferation of Internet of Things (IoT) devices across farms and greenhouses is generating data volumes that increasingly exhibit big data characteristics — high volume, velocity, and variety — creating new challenges for storage, processing, and real-time analysis [15]. Although big data analytics has advanced substantially in other industries, its application to agriculture remains comparatively limited, with data management infrastructure identified as a key bottleneck [8].

Precision agriculture platforms continuously ingest high-frequency sensor telemetry from distributed field devices measuring volumetric water content (VWC), temperature (°C), and electrical conductivity (EC), to support real-time monitoring, irrigation scheduling, and yield optimisation. Such platforms collect data from sensors using protocols such as Message Queuing Telemetry Transport (MQTT), route it through edge or cloud tiers, and store it for both immediate operational queries and long-term analytical use [17]. As the number of connected devices in agricultural IoT grows and sensor transmission intervals shorten, the volume of telemetry flowing into the data layer increases proportionally [18]. This growth is not hypothetical: the broader trend towards higher sensor density per cultivated area is well documented in the precision agriculture literature [18]. This study originates from a software engineering internship at a precision agriculture IoT company whose platform already operates in this regime. The research motivation is proactive: rather than waiting for scaling pressure to become a production constraint, the goal is to identify which architectural pattern best addresses read/write contention *before* the ceiling is hit, and to produce evidence that generalises beyond any single company's deployment.

The data layer of these platforms faces structural tension. A single general-purpose document store such as MongoDB, being the most common choice in small and medium IoT deployments, must simultaneously handle high-volume append writes from sensors and serve low-latency read queries for dashboards, alerts, and historical analytics. As data volume scales, these two workload classes compete for the same resources: write locks delay reads; index maintenance for read performance slows writes. Two architectural patterns from the distributed systems literature propose to address this contention by separating the read and write paths. Command Query Responsibility Segregation (CQRS), as described by Fowler and Young [6, 16], decouples the write model from the read model, allowing each to be optimised independently. The Lambda Architecture, as formalised by Marz and Warren [10], extends this separation into batch and speed layers that process historical and recent data in parallel. Whether these patterns justify their additional operational complexity at the scale of agricultural IoT workloads remains an open question. Kreps [9] argues that Lambda's dual-codebase requirement introduces overhead that simpler stream-based alternatives can avoid, but the trade-off between architectural complexity and performance has not been empirically evaluated for this class of system.

Existing empirical work on Lambda Architecture evaluates data-processing accuracy in adjacent sensor domains [4], but does not benchmark alternative patterns such as CQRS against a single-store baseline, nor does it specifically characterise agricultural IoT read/write workloads.

This study addresses that gap through a controlled benchmark of three MongoDB-based architectural prototypes — a single-store baseline, a CQRS variant with event-driven materialisation of read-optimised aggregates, and a Lambda Architecture variant with batch and speed layers — evaluated under a simulated precision agriculture sensor workload at three increasing scale levels (1,000

to 50,000 sensors). The primary contribution is an empirical comparison that identifies the data volume thresholds at which architectural separation of read and write paths yields measurable benefits in write throughput and query latency, while accounting for the operational complexity each pattern introduces. The benchmark workload and evaluation framework are grounded in documented agricultural telemetry access patterns rather than synthetic or domain-agnostic benchmarks.

1.1 Research Questions

The main research question that guides this paper is the following:

> *How does the performance of CQRS and the Lambda Architecture compare with that of a single-store baseline in terms of write throughput, query latency, and operational complexity when handling increasing data volumes in agricultural IoT telemetry workloads?*

This question is decomposed into four sub-research questions:

Sub Research Question 1: *What are the write throughput and query latency characteristics of a single-store MongoDB baseline under agricultural IoT telemetry workloads at increasing data volumes?*

Without a characterised baseline, no claim about architectural separation can be evaluated. There is no standardised benchmark for this workload class [1, 4]. This sub question establishes the performance profile of the simplest viable architecture — a single MongoDB instance handling both reads and writes — across three data volume levels (1,000 to 50,000 sensors). The baseline characterisation serves two purposes: it provides the reference point against which the CQRS and Lambda variants are compared, and it identifies the scale thresholds at which the single-store architecture begins to exhibit measurable performance degradation. Eyada et al. [5] have demonstrated that MongoDB outperforms relational alternatives for IoT data management, but their evaluation does not characterise how a single-store MongoDB deployment behaves as the write volume increases by orders of magnitude.

Sub Research Question 2: *Under what data volume conditions, if any, does the CQRS variant outperform the single-store baseline on write throughput and query latency?*

CQRS is only warranted where performance benefits are demonstrable [6]; this question tests whether that condition holds for agricultural IoT telemetry. The CQRS prototype separates the write path (raw telemetry ingestion) from the read path (pre-computed aggregates materialised by a background worker consuming from a Kafka topic). This sub-question measures whether eliminating on-demand aggregation from the read path reduces query latency, and whether the asynchronous write-side processing improves or degrades write throughput compared to the baseline. Overeem et al. [11] report that CQRS is increasingly adopted in industry for its reliability and scalability, but their characterisation is qualitative; no empirical performance benchmark of CQRS against a single-store baseline exists for any IoT workload class.

Sub Research Question 3: *Under what data volume conditions, if any, does the Lambda Architecture variant outperform the single-store baseline on write throughput and query latency?*

Kreps [9] argues that Lambda's operational complexity is not universally justified, but this claim has not been empirically tested for this workload class. The Lambda prototype extends the separation of read and write paths by introducing two distinct processing layers: a batch layer that periodically recomputes aggregates from the full data history, and a speed layer (Redis) holding the latest reading for each sensor under a 60-minute expiry. This sub-question measures whether the dual-layer approach provides latency or throughput advantages over both the single-store baseline and the CQRS variant, and at what data volume these advantages, if any, become measurable. Demirezen and Navruz [4] provide the most recent empirical evaluation of Lambda Architecture, but their study focuses on data processing accuracy rather than throughput or latency, and explicitly excludes comparison with alternative architectures.

Sub Research Question 4: *How does operational complexity differ between the three architectural variants?*

Performance gains cannot be evaluated without accounting for their complexity cost. In this study, operational complexity is operationalised along four dimensions: the number of distinct infrastructure components deployed, the volume of deployment configuration required to declare them, the volume of prototype-specific application code required to maintain the read model, and the number of independent write paths whose contents must be kept semantically consistent with one another. This sub-question provides the structural counterpart to the quantitative performance metrics of RQ1–RQ3: an architecture that delivers marginally better throughput but requires substantially more infrastructure may not be justified for a given deployment scale. No standardised metric for operational complexity exists in the literature; the operationalisation defined here is therefore part of the study's contribution.

1.2 Structure

The remainder of this thesis is organised as follows. Section 2 provides background on agricultural IoT telemetry, MongoDB, CQRS, and Lambda Architecture. Section 3 surveys related work on architectural benchmarking and identifies the research gap. Section 4 describes the research methodology and its positioning within the design science paradigm. Section 5 presents the treatment design: the benchmark schema, query types, workload model, technology stack, and evaluation metrics shared by all three prototypes. Section 6 presents the implementation of the three prototypes. Section 7 reports the benchmark results. Section 8 discusses findings, limitations, and implications. Section 9 concludes the thesis.

2 BACKGROUND

This section provides the technical context required to understand the benchmark design and the architectural patterns under evaluation. It describes the domain of precision agriculture IoT telemetry, the document store technology used in all three prototypes, and the two architectural patterns compared with the single-store baseline.

2.1 Agricultural IoT Telemetry

Precision agriculture platforms deploy sensor networks across growing areas to continuously monitor environmental conditions. In a typical soil-monitoring deployment, sensors are buried at multiple depths beneath each cultivation zone and transmit readings of volumetric water content (VWC), soil temperature, and electrical conductivity (EC) at regular intervals — commonly every 10 to 15

minutes [17]. Additional sensor types may include air temperature, humidity, CO_2 concentration, pH, and solar radiation probes, depending on the crop and growing environment.

The data layer of such a platform serves two distinct workload classes. The write path must ingest sensor readings at a sustained rate proportional to the number of deployed devices and their transmission frequency. A platform with 10,000 sensors transmitting every 15 minutes produces approximately 11 write operations per second at steady state, with the load distributed unevenly due to staggered transmission offsets. The read path must serve queries that range from point lookups (retrieving the latest reading for a single sensor) to analytical aggregations spanning weeks or months of historical data (trend analysis across an entire cultivation zone). These two workload classes differ in their I/O profile: writes are append-only and uniform in document size, whereas reads vary by orders of magnitude in the volume of data they scan [18].

The spatial organisation of a precision agriculture deployment follows a hierarchical structure. A zone represents a named physical area within the growing facility — a greenhouse section, a field block, or a planting row. A cultivation represents the specific crop programme assigned to a zone, including what is being grown and what irrigation profile applies. A sensor is the individual physical device buried in the soil within a cultivation. This zone–cultivation–sensor hierarchy determines how data is queried: operational dashboards query by zone, agronomic analysis queries by cultivation, and device diagnostics query by individual sensor.

2.2 MongoDB as a Document Store

MongoDB is a general-purpose document-oriented database that stores data as BSON (Binary JSON) documents within collections. It supports flexible schemas, horizontal scaling through sharding, and a rich aggregation pipeline for server-side data transformations. These properties make it a common choice for IoT platforms at small-to-medium scale, where schema flexibility accommodates evolving sensor payloads and the aggregation pipeline provides ad-hoc analytical capability without a separate analytics engine.

In the context of this study, MongoDB serves as the storage engine across all three prototype architectures. The baseline prototype (P1) uses MongoDB as a single store: all sensor readings are written to a single collection, and all queries are computed at request time using the aggregation pipeline against the raw data. This design is representative of how many IoT startups operate in their early stages — a single database handling both the write and read workloads. As data volume grows, this configuration faces increasing resource contention: the indexes required to serve low-latency reads impose maintenance overhead on writes, while large analytical scans compete with ingest operations for available I/O bandwidth.

Figure 1 illustrates this configuration. Sensor readings arrive via an MQTT broker and are persisted to `telemetry_raw`, a single append-only collection that stores every reading as a fully denormalised document. All read queries — from point lookups of the latest sensor value to aggregations spanning an entire growing season — are then computed at request time directly against this same collection using MongoDB's aggregation pipeline. The key feature of the diagram is its simplicity: the single `telemetry_raw` box serves both the write path (left arrow) and the read path (right arrow), with no intermediate processing or separate storage. This is both the architecture's strength — there is nothing else to deploy, configure, or keep in sync — and its weakness, because every read query, regardless of whether it needs one document or tens of thousands, competes with ingest writes for the same collection and the same set of indexes.

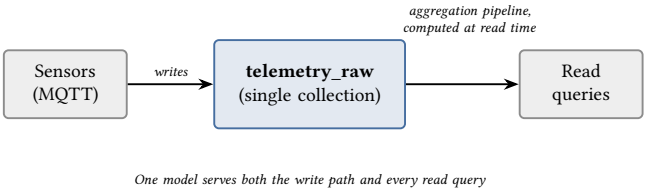


Figure 1: P1: MongoDB as a single store, with all query types computed at request time directly against `telemetry_raw`.

By using MongoDB across all three prototypes, the benchmark isolates the architectural pattern as the independent variable. Any observed performance differences are attributable to how data flows through the system — not to differences in the underlying storage engine.

2.3 CQRS

Command Query Responsibility Segregation (CQRS) is an architectural pattern that uses separate models for reading and writing data. In its canonical formulation, Fowler [6] describes CQRS as the recognition that the data model optimised for writes (commands) is often structurally different from the model optimised for reads (queries). Rather than forcing both workloads through a single shared model, CQRS separates them, allowing each to be designed and scaled independently.

Young [16] elaborates on the mechanics of this separation. The write side accepts commands, validates them, and persists them to a store. A separate process, often asynchronous, reads from this store and materialises one or more read-optimised views that serve the query workload. The read models are derived data: they are projections of the write-side data shaped for specific query patterns. This materialisation step introduces eventual consistency: the read model may lag behind the write model by the time required to process and project new data.

In the context of this study, the CQRS architecture implements this separation as follows: sensor telemetry is written to a raw data collection (the write model), published to a Kafka topic, and consumed by a background worker that continuously materialises pre-computed aggregates into a separate collection (the read model). Read queries are served from the pre-computed aggregates rather than computed on the fly from raw data, as in the baseline. Figure 2 illustrates this data flow.

As the figure shows, `telemetry_raw` serves as the write model, storing every sensor reading unchanged. Each new document is also published to a Kafka topic, from which a continuously running Go worker consumes events and materialises pre-computed aggregates into `telemetry_aggregated`, the read model. The critical visual distinction is on the right side of the diagram: the read path no

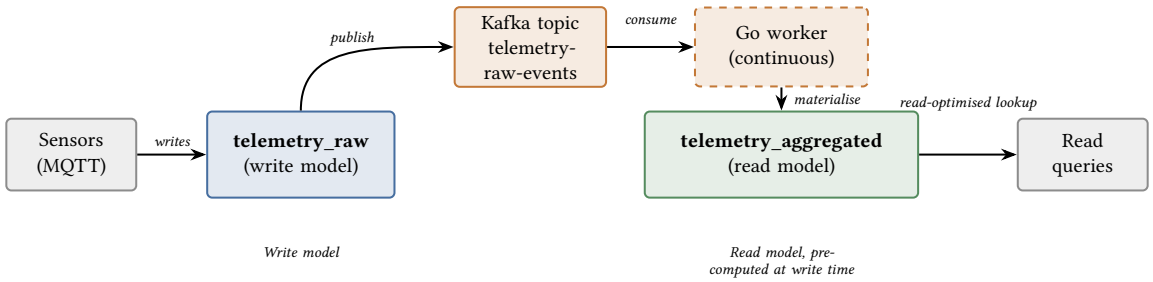


Figure 2: P2: CQRS separation of the write model (`telemetry_raw`) from the read model (`telemetry_aggregated`), materialised continuously by a Kafka-consuming Go worker.

longer touches `telemetry_raw` at all. Instead, read queries retrieve pre-computed results directly from the separate `telemetry_aggregated` collection, with no aggregation pipeline executed at request time. This is the CQRS separation made visible: the write model and the read model are physically distinct collections, maintained by different processes, so that the read workload is decoupled from the write workload by construction.

Fowler [6] cautions that CQRS introduces complexity and should only be applied to portions of a system where it provides demonstrable benefit. This caution motivates the central research question of this thesis: under what conditions does the benefit materialise for agricultural IoT workloads?

2.4 Lambda Architecture

Lambda Architecture, formalised by Marz and Warren [10], is a data processing architecture designed to handle both historical and real-time data through three layers: the batch layer, the speed layer, and the serving layer.

The batch layer stores the complete, immutable dataset and periodically recomputes aggregate views from the full history. Because it processes all data from scratch on each run, the batch layer is self-correcting: any errors in previous computations are overwritten by the next complete pass. The speed layer compensates for the batch layer's latency by processing only the most recent data in near-real-time, producing approximate views that cover the window between the last completed batch run and the present moment. The serving layer merges the outputs of the batch and speed layers at query time, presenting a unified view to the client.

The architecture's central design principle is data immutability: the raw data is never modified in place, and all derived views are recomputable from the original input. This property provides fault tolerance — a corrupted view can always be rebuilt — at the cost of maintaining two separate processing codebases (batch and speed) that must produce semantically compatible outputs.

Kreps [9] critiques this dual-codebase requirement as a significant operational burden. He argues that a single stream processing system capable of replaying historical data (a pattern he terms the Kappa Architecture) can achieve equivalent correctness guarantees with substantially less infrastructure. This debate remains theoretical: neither side has produced empirical evidence comparing the architectures under controlled conditions for a specific workload class.

In the context of this study, the Lambda Architecture prototype implements the batch layer as a scheduled Go worker that periodically recomputes aggregates from the full raw data history using MongoDB aggregation pipelines. The speed layer is implemented as a Redis cache covering the most recent reading for each sensor, held under a 60-minute expiry. Read queries merge the batch and speed layer outputs at the API level. Figure 3 illustrates this split.

As the figure shows, `telemetry_raw` serves as the immutable master dataset from which the batch worker periodically recomputes `telemetry_aggregated`. A separate Kafka-fed writer maintains the Redis speed layer with the most recent reading per sensor. Unlike P2, which has a single read model, the diagram here shows two separate read paths: the upper path (green, batch layer) serves aggregation queries over historical windows from pre-computed views, while the lower path (purple, speed layer) serves the latest-reading lookup directly from Redis. This dual-path structure is the source of the Lambda Architecture's additional operational surface: the raw insert, the batch recomputation, and the speed-layer write are three independently maintained write paths whose outputs must remain semantically consistent with one another.

3 RELATED WORK

This section surveys prior work across four dimensions — domain literature, theoretical foundations, database and architecture evaluations, and empirical benchmarking methodology — and identifies the specific gap this thesis addresses.

3.1 Domain Literature

Zamora-Izquierdo et al. [17] provide a detailed study of a real smart farming deployment built on the FIWARE platform, establishing the sensor types, communication protocols, and three-tier architecture (cyber-physical, edge, cloud) characteristic of the domain. Their cloud layer implicitly absorbs both current-state reads and historical data queries without architectural separation of read and write paths. Zhang et al. [18] survey edge computing applications in agricultural IoT, confirming the growth trajectory of data volumes and the scalability challenges that arise in cloud-centric architectures as sensor density increases. Kamilaris et al. [8] review 34 studies on big data analysis in agriculture, finding that although the data volumes generated by modern farming operations increasingly exhibit big data characteristics, analytical infrastructure — and in particular

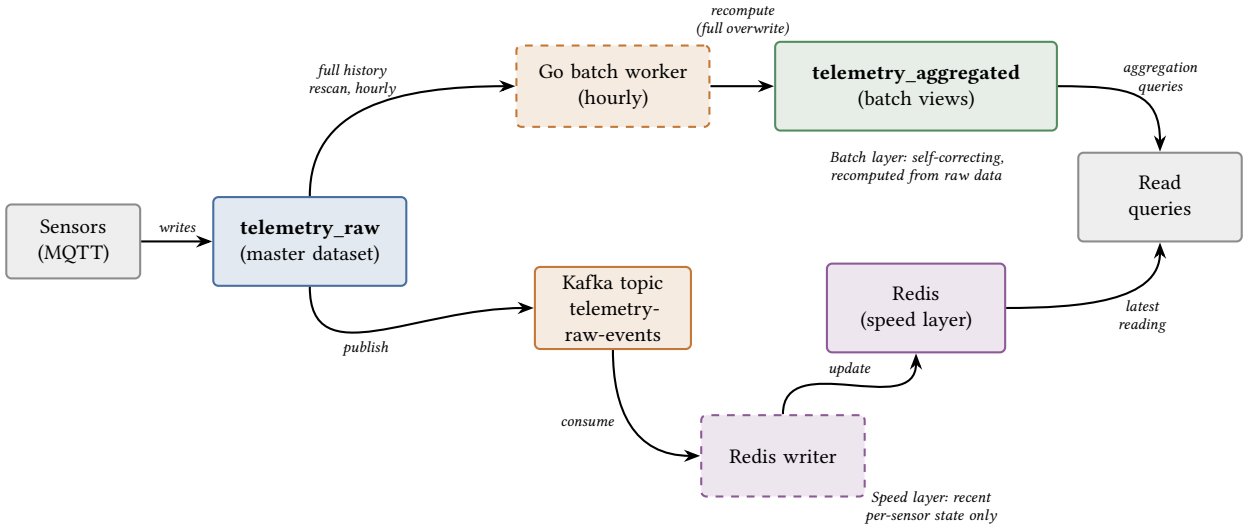


Figure 3: P3: Lambda Architecture split into a batch layer (hourly recomputation of `telemetry_aggregated` from the full raw history, serving aggregation queries) and a speed layer (Redis, serving the latest-reading lookup), merged at the API into a single serving layer.

the data platform architectures that support it — remain underdeveloped compared to other industries. All three papers establish the domain context but none engages with data platform architecture selection as a research problem.

3.2 Theoretical Foundations

Fowler [6] and Young [16] together constitute the primary reference basis for CQRS: Fowler provides the conceptual definition and its applicability conditions, cautioning that the pattern is only warranted where the performance benefit is demonstrable; Young describes the operational mechanics of maintaining separate read and write stores. Marz and Warren [10] provide the definitive specification of Lambda Architecture's three-layer model (batch, speed, serving), designed for systems that must support both historical throughput and real-time query latency. Kreps [9] challenges Lambda's default adoption in a widely cited practitioner article, arguing that its dual-codebase requirement introduces significant operational overhead and that a simpler single-stream alternative (which he terms the Kappa Architecture) can achieve equivalent guarantees with less complexity. Taken together, these sources establish that both CQRS and Lambda Architecture are theoretically motivated for mixed read/write systems, while also demonstrating that the case for either pattern over a well-tuned single-store baseline requires empirical, not merely theoretical, justification.

3.3 Database and Architecture Evaluations

Eyada et al. [5] provide an empirical performance evaluation of MongoDB versus MySQL for IoT data management across different cloud environments, demonstrating that MongoDB outperforms relational alternatives on write throughput and read latency for IoT workloads at increasing data volumes. Their study validates MongoDB as an appropriate storage engine for IoT telemetry, but evaluates only the choice of database technology, not the choice of architectural pattern built on top of it. Whether a single MongoDB instance or a multi-component architecture (CQRS, Lambda) better serves the mixed read/write workload remains outside their scope.

Overeem et al. [11] present the first empirical characterisation of event sourced systems — a pattern closely related to CQRS — based on interviews with 25 engineers across 19 production systems. Their study identifies reliability, flexibility, and scalability as the primary motivations for adopting event sourcing, and documents five recurring challenges including schema evolution and steep learning curves. However, their methodology is qualitative (grounded theory); the study does not measure throughput, latency, or any quantitative performance metric, nor does it compare event-sourced architectures against a single-store baseline.

3.4 Empirical Benchmarking Precedents

Comparative architectural benchmarking through controlled experiments is an established methodology in computer science. Andreadis et al. [1] demonstrate this approach at VU Amsterdam, proposing and validating a reference architecture for datacenter scheduling through a survey-design-prototype-experiment pattern — the same methodological structure this thesis follows. Vinken [13] applies this approach at thesis level at the University of Amsterdam, benchmarking competing virtualisation architectures for distributed web applications using Docker-based environments and synthetic workloads, establishing that controlled architectural comparison is an accepted thesis-level contribution. Hesse [7] extends empirical architectural benchmarking to stream processing systems in a European doctoral thesis, including sensor telemetry workloads, and demonstrates that synthetic-but-realistic workloads are a methodologically accepted substitute for production data in this class of evaluation.

The most recent empirical evaluation of Lambda Architecture in a sensor domain is by Demirezen and Navruz [4], who benchmark a Lambda deployment for ADS-B air-traffic surveillance data. Their study introduces a methodology for testing Lambda's data-processing accuracy under adverse ingestion conditions — including delayed data, repeated data, and batch layer failure — and provides the first empirical evidence for Lambda's eventual consistency properties. Critically, however, their evaluation targets processing accuracy rather than write throughput or query latency, and their scope statement explicitly excludes IoT applications and comparison with alternative architectures. Their survey of previous Lambda deployments (Table A1 in the original article) spans social media, scientific monitoring, and agriculture, but none of the surveyed works benchmarks CQRS alongside Lambda, none uses a single-store baseline for comparison, and none characterises agricultural IoT read/write workloads as the object of evaluation.

3.5 Cross-Domain Architectural Performance Comparisons

Beyond the domain-specific and methodology-focused studies surveyed above, several recent works apply controlled benchmarking to compare competing architectural patterns in non-agricultural contexts. Although none addresses agricultural IoT workloads, they validate the methodological approach of isolating architectural pattern as an independent variable and measuring performance under controlled conditions.

Blinowski et al. [3] benchmark monolithic versus microservice architecture in a web-application context, measuring performance and scalability under increasing load. Their study demonstrates that the trade-off between a simpler, unified architecture and a decomposed, multi-component alternative is an empirical question that controlled benchmarking can answer — a principle this thesis applies to the single-store versus CQRS and Lambda comparison.

Van Landuyt et al. [12] present a comparative performance evaluation of multi-model NoSQL databases against polyglot persistence — the practice of using multiple specialised data stores rather than a single general-purpose store. Their comparison between a single multi-model database serving all access patterns and a polyglot configuration with separate stores for separate workloads directly parallels the architectural question this thesis addresses: whether separating read and write paths into distinct components yields measurable performance benefits over a single-store approach.

Avritzer et al. [2] propose a domain-based methodology for assessing the scalability of microservice deployment configurations through operational profiles and load testing. Their systematic use of controlled load profiles to compare architectural deployment alternatives informs the workload modelling approach adopted in this study.

These three studies confirm that controlled architectural benchmarking is an active and accepted methodology across multiple software engineering domains. The absence of equivalent work targeting agricultural IoT data architectures reinforces the gap identified in Section 3.7.

3.6 Summary of Related Work

Table 1 summarises the literature surveyed above along the four dimensions that define this thesis's contribution. Each row represents a prior study, characterised by its application domain, the architectures it evaluates, the evaluation metrics it reports (if any), whether it includes a baseline comparator, and whether it targets agricultural IoT workloads specifically. The final two columns highlight the two conditions that are individually common but jointly absent from the literature: baseline comparison and agricultural IoT focus. The empty intersection — an empirical benchmark of CQRS and Lambda against a single-store baseline for agricultural IoT read/write workloads — constitutes the research gap addressed in Section 3.7.

3.7 Research Gap

The gap this thesis addresses lies at the intersection of four dimensions: (1) empirical benchmarking rather than theoretical argument, (2) inclusion of CQRS alongside Lambda Architecture, (3) comparison against a single-store baseline, and (4) characterisation of agricultural IoT read/write workloads specifically. No published work identified in the literature surveyed above addresses this intersection. This thesis provides the first controlled, empirical comparison of these three architectural approaches under a simulated precision agriculture sensor workload at increasing data volumes.

4 METHODOLOGY

This section presents the research methodology, the benchmark design, and the evaluation metrics used to answer the research questions defined in Section 1.1. The study is conducted in the context of an internship at Agurotech, a precision agriculture IoT company, where its findings inform an internal evaluation of possible new data architectures for the company's sensor telemetry platform. Consistent with this practical motivation, the benchmark schema and query set defined below are nonetheless kept generic: they are framed so that the contribution generalises to any precision agriculture IoT platform with a comparable read/write workload, rather than being tied to implementation details specific to Agurotech's production system.

4.1 Research Methodology

The research questions formulated in Section 1.1 are empirical in nature: they ask how measurable properties of a system change when its architecture is varied under controlled conditions. This places the study within the design science paradigm as characterised by Wieringa [14], in which the researcher designs one or more artifacts — in this case, three prototype data architectures — and validates them through systematic investigation in a representative context.

The core of the study is a controlled benchmark comparison. Three prototype architectures are constructed for the same agricultural IoT telemetry workload: a single-store MongoDB baseline (P1), a CQRS variant with event-driven materialisation via Kafka (P2), and a Lambda Architecture variant with a batch worker and Redis speed layer (P3). All three prototypes share the same storage engine, ingest API, synthetic dataset, and containerised deployment environment. The architectural pattern is thereby isolated as the sole independent variable; differences in measured performance

Table 1: Summary of related work across the four dimensions of the research gap.

Study	Domain	Architectures	Eval. Metrics	Baseline?	Agri-IoT?
Zamora-Izquierdo et al. [17]	Smart farming	Single-tier cloud	Deployment valid.	No	Yes
Zhang et al. [18]	Agricultural IoT	Edge/cloud survey	Survey only	No	Yes
Kamilaris et al. [8]	Agriculture	Big data review	Survey only	No	Yes
Eyada et al. [5]	IoT general	MongoDB vs. MySQL	Latency, throughput	Yes (MySQL)	No
Overeem et al. [11]	Enterprise	Event sourcing / CQRS	Qualitative	No	No
Demirezen & Navruz [4]	ADS-B surv.	Lambda only	Accuracy	No	No
Andreadis et al. [1]	Datacenter	Reference arch.	Methodology	Yes	No
Vinken [13]	Web apps	Virtualisation	Latency, throughput	Yes	No
Hesse [7]	Stream proc.	Processing engines	Throughput, latency	Yes	No
Blinowski et al. [3]	Web apps	Monolithic vs. microservice	Performance, scalability	Yes (monolithic)	No
Avritzer et al. [2]	Microservices	Deployment configs	Scalability	Yes	No
Van Landuyt et al. [12]	Database	Multi-model vs. polyglot	Performance	Yes (single-model)	No

can be attributed to the pattern rather than to incidental implementation choices. This controlled-comparison approach follows the methodology established by Andreadis et al. [1] for architectural evaluation at VU Amsterdam and by Vinken [13] for Docker-based benchmarking of distributed systems at thesis level.

The prototypes are evaluated using synthetic workloads that simulate realistic sensor telemetry at three scale levels (1,000 to 50,000 sensors). Synthetic-but-realistic workloads are a methodologically accepted substitute for production data in systems benchmarking, as demonstrated by Hesse [7] for stream processing architectures under sensor telemetry loads. The use of synthetic data allows full reproducibility: the same deterministic dataset is replayed against each prototype, ensuring that observed differences are not artifacts of data variation.

Following Wieringa's engineering cycle [14], the study proceeds through problem investigation (the literature survey and gap identification in Sections 2–3), treatment design (the three prototypes and the benchmark schema defined in this section), and treatment validation (the controlled experiments reported in Section 7). Figure 4 illustrates this design cycle and its relationship to the two phases that fall outside the scope of the study.

Two distinct scoping decisions follow from this. First, the design cycle is executed as a single pass: the three prototypes are validated once against the benchmark, and no prototype is redesigned in response to the resulting measurements. The corrective work performed during the study — for example, the query-anchoring fix described in Section 7.1 — addresses defects in the benchmark harness rather than in the treatments themselves, and therefore does not constitute a second iteration of treatment design. Second, treatment implementation and implementation evaluation, the two phases that follow the design cycle in Wieringa's model, are out of scope for a separate reason: they require introducing the treatment into the real problem context — here, deployment into Agurotech's production platform — and evaluating its effect there. This thesis does not undertake that step. Consistent with this, the prototypes are implemented as self-contained Docker Compose configurations designed for reproducibility rather than operational deployment (Section 6).

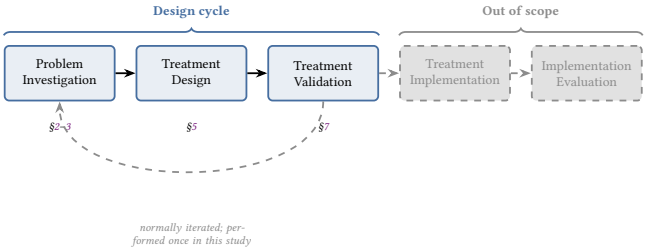


Figure 4: Wieringa's engineering cycle [14], showing the three design-cycle phases performed once in this study and the two downstream phases excluded from its scope.

Section 5 now documents the treatment design phase of the engineering cycle: the concrete decisions that translate the three architectural patterns motivated in Section 2 into an executable, comparable benchmark.

5 TREATMENT DESIGN

This section documents the treatment design phase of the engineering cycle: the concrete decisions that translate the three architectural patterns motivated in Section 2 into an executable, comparable benchmark. Wieringa's model treats treatment design as specifying not only the artifact but also how it will be validated, so this phase produced five interlocking decisions, each detailed in its own subsection below: the benchmark schema shared by all three prototypes (Section 5.1), the five query types that exercise it (Section 5.2), the workload model that scales the dataset across sensor counts (Section 5.3), the technology stack common to all three prototypes (Section 5.4), and the metrics by which treatment validation is subsequently carried out (Section 5.5). Designing the validation approach alongside the artifacts themselves, rather than as an afterthought once the prototypes existed, is what allows Section 7 to apply an identical measurement protocol to all three prototypes without retrofitting.

5.1 Benchmark Schema

This subsection defines the data model shared by all three prototypes: the three MongoDB collections that form the benchmark's storage layer, their document structure, and the indexes that serve the benchmark query types. The benchmark uses three MongoDB collections, each with a distinct purpose and access pattern. All three prototypes share the same collections; the prototypes differ only in how data flows through the system, not in how it is stored.

device_registry. A lookup table seeded once before each benchmark run and never modified during it. One document is created per simulated sensor, storing the sensor's serial number, GPS coordinates, zone assignment, and cultivation assignment. When the ingest API receives an MQTT message, it uses the sensor's serial number to look up this collection and retrieve the zone and cultivation context needed to construct a fully enriched `telemetry_raw` document. The device registry is the authoritative source of truth for sensor metadata; GPS coordinates and cultivation identifiers transmitted in the MQTT payload are ignored in favour of the registered values.

telemetry_raw. The primary time-series store. Every sensor reading creates a new document in this collection; the collection is append-only and no document is ever updated or deleted. Each document is fully denormalised: it embeds zone and cultivation metadata alongside the sensor readings, eliminating the need for join operations at query time. This design choice ensures that the P1 baseline does not incur hidden join costs that would bias the comparison against a single-store architecture (Section 2).

Each document contains two timestamp fields: `timestamp` (device-generated event time) and `ingestedAt` (API-generated processing time). Their difference constitutes the ingest lag metric used in RQ1. Sensor readings are recorded at four soil depths (15, 30, 45, and 60 cm), each containing volumetric water content (VWC), temperature, and electrical conductivity (EC) values.

Three compound indexes are defined on this collection to serve the five benchmark query types:

- `{sensor.serial:1, timestamp:-1}` — Q1 and Q2;
- `{zone.zoneId:1, timestamp:-1}` — Q3;
- `{cultivation.cultivationId:1, timestamp:-1}` — Q4 and Q5.

In each index the equality-filtered field precedes the time-range field so that MongoDB can satisfy both the filter and the sort from the index without an in-memory sort step. These indexes are part of the baseline optimisation: a legitimate single-store comparator must receive every reasonable, standard optimisation, or the comparison with P2 and P3 would be academically indefensible.

telemetry_aggregated. A pre-computed summary store used in P2 and P3 only; not present in P1. Background workers write pre-computed aggregates to this collection so that read queries can retrieve results directly rather than computing them from raw data at request time. The collection contains three document types: `zone_daily` (one document per zone per day, serving Q3), `cultivation_daily` (one document per cultivation per day, serving Q4 and the daily component of Q5), and `cultivation_180d_rollup` (one document per cultivation, serving the full-period summary of Q5). Each document stores the already-computed aggregate value; no raw readings are stored in this collection and no aggregation pipeline is invoked at query time in P2 or P3. This is the architectural distinction under test: P1 computes at read time; P2 and P3 compute at write time.

The spatial hierarchy used throughout the benchmark is zone → cultivation → sensor. The farm entity that exists above zones in production systems is excluded because none of the five benchmark query types require farm-level grouping; including it would add referential complexity without affecting any measured variable.

5.2 Benchmark Query Types

The five benchmark queries span the full range of temporal access patterns present in precision agriculture IoT platforms, from sub-second point lookups to full-season analytical scans. They are ordered from least to most expensive for the single-store baseline (P1), so that the performance gap between P1 and the pre-computation approaches of P2 and P3 becomes progressively more visible.

Q1 — Latest reading per sensor (point lookup). Retrieve the single most recent document for a given sensor serial number. This query has no time window and returns exactly one document. It represents the most common operational access pattern: checking current soil conditions for a specific sensor. In P1, it is served by a descending index lookup; in P2, by a single-document read from `latest_by_sensor` in `telemetry_aggregated`; in P3, exclusively from Redis.

Q2 — Sensor time-range (24 hours). Retrieve all readings for a specific sensor over the last 24 hours, returning up to 96 documents (one reading every 15 minutes). This query tests indexed range scan efficiency and represents a farmer reviewing yesterday's sensor behaviour to check for anomalies or irrigation response.

Q3 — Zone aggregation (7 days). Compute the average VWC across all sensors in a given zone over the last 7 days, grouped by day. A zone contains one active cultivation with 1–5 sensors. In P1, this requires a MongoDB aggregation pipeline filtering by `zone.zoneId` and timestamp range; in P2 and P3, the daily zone-level averages are pre-computed and served directly from `telemetry_aggregated`.

Q4 — Cultivation historical trend (30 days). Compute the daily average soil temperature at all four depths for a specific cultivation over the last 30 days, producing a 30-point trend line. In P1, the aggregation pipeline processes up to 14,400 documents (96 readings per sensor per day × up to 5 sensors × 30 days). In P2 and P3, daily averages are materialised by the background worker.

Q5 — Full cultivation period VWC summary (up to 180 days). Compute the average VWC at all four depths across the entire cultivation period for a specific cultivation. The cultivation period is simulated at up to 180 days. Per cultivation, the query returns 180 daily-average VWC values plus one overall 180-day rollup average per depth (4 depths × 181 values = 724 values total). All computation is performed server-side in every prototype to ensure the only variable under test is *when* aggregation occurs (read time vs. write time), not *where* (server vs. client). In P1, this pipeline may process up to 86,400 documents; in P2 and P3, the result is a single-document lookup.

5.3 Workload Model

Sensor count is the single independent variable. All other parameters — cultivation distribution, zone count, query window lengths, transmission interval, and cultivation period — are held constant across scale levels. This isolates the effect of data volume on each architecture.

Scale levels. The benchmark is run at three sensor counts: 1,000, 10,000 and 50,000. The corresponding sustained ingest rates are approximately 1.1, 11.1 and 55.6 writes per second, respectively. The data generator seeds a full 180-day history at each scale level, so that Q5's 180-day window is covered in its entirety; `telemetry_raw` consequently holds approximately 17.3 million documents at 1,000 sensors, 173 million at 10,000, and 865 million at 50,000 (96 readings per sensor per day over 180 days). A 30-day span of that history, approximately 2.88 million documents at 1,000 sensors and 144 million at 50,000, corresponds to the window Q4 aggregates over.

Sensor and cultivation distribution. Sensor-per-cultivation counts are assigned using a weighted random distribution seeded deterministically from the cultivation identifier: 70% of cultivations have 1 sensor, 20% have 2, 7% have 3, and 3% have 5. This produces an average of approximately 1.4 sensors per cultivation, scaling the number of cultivations and zones proportionally with sensor count: approximately 700 zones at 1,000 sensors to 35,000 zones at 50,000 sensors. Each cultivation is assigned to exactly one zone, and each zone contains one active cultivation.

Transmission model. Each sensor transmits one reading every 15 minutes. Transmission offsets are staggered deterministically from each sensor's serial number so that sensors do not transmit simultaneously, simulating realistic deployment conditions where sensor clocks are not perfectly synchronised.

Cultivation period and benchmark window. The maximum simulated cultivation period is 180 days, representing a realistic growing season. Sowing dates are staggered across cultivations so that the dataset contains cultivations at different lifecycle stages. To support Q5's 180-day history requirement, the synthetic data generator seeds `telemetry_raw` with the full 180 days of history, ending at a fixed reference date to which the query harness is subsequently anchored (Section 7.1). The query workload is therefore evaluated against this seeded history. The separate live replay used for the ingest measurements runs for minutes rather than days, since its purpose is to observe the ingest path in steady state rather than to extend the history the queries read; its duration per prototype and scale level is reported in Section 7.5.

5.4 Technology Stack

All three prototypes share the same ingest layer; differences between prototypes exist only in the background processing and query layers. Table 2 summarises the components. The first three entries — the MQTT broker, the ingest API, and MongoDB — are shared across all three prototypes and form the common ingest path described in Section 6.1. The message broker (Kafka), background worker, and speed layer (Redis) are present only in the prototypes whose architectural pattern requires them: Kafka and the worker in both P2 and P3, Redis in P3 only. The benchmark harness and data generator run on the host machine rather than inside Docker, so that measurement overhead from container scheduling does not contaminate the latency and throughput figures.

Table 2: Implementation stack components.

Component	Technology	Purpose
MQTT broker	Mosquitto	Receives sensor transmissions
Message broker	Kafka (KRaft)	Decouples ingest from consumers (P2, P3)
Ingest API	Node.js	MQTT subscriber, device lookup, document construction
Database	MongoDB 7.0	Stores all three collections
Background worker	Go	Continuous materialisation (P2); hourly batch (P3)
Speed layer	Redis	Latest reading per sensor, 60-min expiry (P3 only)
Benchmark harness	Go	Measures query latency and ingest lag
Data generator	Go	Seeds history and replays live payloads

The three prototypes are deployed as separate Docker Compose configurations, each with its own named volumes to prevent data leakage between runs. P1 deploys Mosquitto and MongoDB only. P2 adds Kafka and a continuous Go worker. P3 adds Kafka, a scheduled Go batch worker, and Redis. The benchmark harness and data generator run on the host machine, outside Docker, to avoid measurement interference from container scheduling.

5.5 Evaluation Metrics

The treatment design produces not only the three prototypes but also the metrics by which they are subsequently compared. Four metric categories are collected, corresponding to the four research sub-questions: sustained write load and ingest lag (Sub-RQ 1), query latency (Sub-RQ 2 and 3), and operational complexity (Sub-RQ 4). Each metric is defined in full, together with its measurement protocol and reporting conventions, in Section 7.2, immediately before the results it is used to interpret.

Section 6 now describes how this treatment design is realised as concrete, executable software: the schema, query types, workload model, and technology stack specified above become the three running prototypes that Section 7 subsequently validates.

6 IMPLEMENTATION

This section realises, as concrete executable software, the treatment designed in Section 5: the three benchmark prototypes (P1, P2, P3), the shared ingest pipeline they build upon, and the synthetic data generator used to populate and drive the benchmark. All three prototypes share a single ingest codebase, selected at deployment time via a `PROTOTYPE` environment variable (`p1`, `p2`, or `p3`); they differ only in the background processing and query layers built on top of this shared foundation, consistent with the isolation of architectural pattern as the sole independent variable (Section 4.1).

6.1 Shared Ingest Pipeline

All three prototypes ingest sensor telemetry through an identical path, illustrated in Figure 5, so that any performance difference

measured downstream can be attributed to what happens after ingestion rather than to how data enters the system.

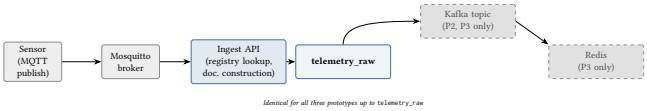


Figure 5: Shared ingest path common to all three prototypes. Solid boxes are present in all configurations; dashed boxes are deployed only in the prototypes indicated.

As the figure shows, a sensor publishes a reading via MQTT to a Mosquitto broker, which delivers it to the ingest API. The API enriches the reading with zone and cultivation context from the `device_registry` collection and writes the resulting document to `telemetry_raw`. Everything to the left of `telemetry_raw` in the diagram — the sensor, the broker, and the API — is byte-for-byte identical across all three prototypes; this is the shared ingest front-end against which the ingest lag metric is measured (Section 7.2). The dashed boxes to the right of `telemetry_raw` represent the prototype-specific divergence point: in P2 and P3, the document is additionally published to a Kafka topic; in P3, a Kafka consumer writes the most recent reading per sensor to a Redis cache. It is this divergence that the benchmark is designed to evaluate.

Each simulated sensor publishes its reading as a JSON payload to a single flat MQTT topic on a Mosquitto broker configured for anonymous access and without message persistence, since payloads are either consumed immediately by the ingest API during a run or reproducible on demand by re-invoking the data generator; there is therefore no benchmark-relevant reason to retain them at the broker after delivery. The payload carries the sensor's serial number, the four-depth VWC, temperature, and EC readings, and a timestamp field set by the publisher at the moment of transmission. This timestamp is not merely descriptive metadata: it is the anchor against which ingest lag is measured, and its presence in the wire payload (rather than being assigned later, at the API) is what allows the benchmark to measure delay incurred in transit and in the broker, rather than only delay internal to the ingest API. The precise extent of the interval this metric covers, and what it therefore excludes, is set out in Section 7.2.

On receiving a message, the ingest API performs three steps. First, it resolves the sensor's serial number against the `device_registry` collection to obtain the zone and cultivation context under which the reading was taken; the registry, rather than the MQTT payload itself, is treated as the authoritative source for this context, so that a device transmitting stale or spoofed location metadata cannot corrupt the benchmark schema. Second, it constructs the resulting `telemetry_raw` document through a single, pure, side-effect-free construction function that takes the raw payload and the registry lookup result as input and returns a complete document as output, with no database or network access of its own. Isolating document construction in this way makes it independently testable: its correctness can be verified through unit tests that assert on the shape and content of the returned document for known inputs, without requiring a running MongoDB or Kafka instance, and without the test outcome depending on timing or I/O. Third, the API persists the document — directly into `telemetry_raw` for P1, or by publishing it onto the `telemetry-raw-events` Kafka topic, partitioned by `sensor.serial`, for P2 and P3. If the registry lookup in the first step fails to find a matching serial number, the API logs the event and discards the message rather than raising an error, on the basis that an ingest path handling continuous device traffic must tolerate occasional unregistered or malformed input without interrupting service to every other sensor.

Access to MongoDB and Kafka is each mediated through a narrow gateway interface, so that the document-construction logic described above has no direct dependency on either client library. This separation of concerns is what permits document construction to be unit-tested in isolation from the infrastructure it eventually writes to, and is revisited in Section 6.6 as part of the broader verification strategy applied before each scale-level run.

6.2 Synthetic Data Generator

A dataset at the scale required by this benchmark (Section 5.3) cannot be produced through real-time MQTT replay alone within a practical amount of time — populating 180 days of history at the 50,000-sensor scale level through simulated real-time transmission would take 180 days to replay in real time. A dedicated Go data generator therefore produces the dataset offline, in two distinct phases implemented as the `seed` and `replay` subcommands of a single binary.

It is worth being precise about what the specific sensor values generated in either phase are, and are not, expected to influence. The generator produces VWC, temperature, EC, and battery readings within fixed, realistic ranges (VWC 15–55 %, temperature 8–28 °C, EC 0.1–1.5 dS/m, battery 60–100 %) so that the resulting documents are domain-plausible and so that any manual inspection of the data during development or debugging reflects a believable deployment. However, none of the metrics reported in this thesis — write throughput, query latency, or ingest lag — depend on which specific value within these ranges a given document holds. Because every reading is stored using the same fixed-width numeric representation regardless of its magnitude, a document containing a VWC reading of 16 % occupies exactly as many bytes, and is exactly as expensive to write, index, and read, as one containing 54 %. What determines every performance metric measured in this study is the *volume* of data — the number of documents, and the total size of the collections and indexes they populate at a given sensor count — not the particular values those documents contain. The realism of the generated values is therefore a property that matters for the plausibility and external validity of the dataset as a stand-in for a real deployment, but is orthogonal to, and does not confound, the benchmark's internal validity.

seed. This subcommand performs the offline pre-population phase. It drops any pre-existing collections, then bulk-inserts `device_registry` and 180 days of historical `telemetry_raw` documents directly into MongoDB, bypassing the MQTT ingest path entirely for this phase; there is no benchmark-relevant reason to route this non-time-sensitive historical population through the same real-time pipeline used during the timed run. Index creation on `telemetry_raw` is deliberately deferred until after the bulk insert completes: building the three compound indexes in a single

pass against an already-populated collection is markedly faster than maintaining them incrementally while several hundred million documents are being inserted, an effect that becomes especially pronounced at the 50,000-sensor scale level. Once created, the indexes are defined to match, field for field and order for order, the definitions already established in the `mongo-init` scripts, so that the pre-populated portion of the dataset produced by `seed` and the portion subsequently written during the timed `replay` run are indexed identically. Sensor serial numbers are generated as zero-padded numeric strings (e.g. `0000001`), and every value drawn for a given sensor is seeded deterministically from that sensor's serial number, so that re-running `seed` against the same sensor count reproduces an identical dataset rather than a new random sample each time — a property required for the reproducibility goals discussed in Section 4.1. At the 1,000-sensor scale level, this phase was verified to insert 17,280,000 documents at a sustained rate of approximately 116,000 documents per second.

`replay`. This subcommand drives the live ingest measurement. It publishes MQTT payloads at the sustained rate implied by the scale level — sensor count divided by 900 messages per second, corresponding to each sensor transmitting once every 15 minutes — for a configurable duration of minutes rather than days, since the purpose of this phase is to observe the ingest path in steady state rather than to add materially to the seeded history, and with each sensor's transmission offset staggered deterministically from its serial number so that sensors do not transmit in lockstep, as described in Section 5.3. The critical distinction between `replay` and `seed` is in how the `timestamp` field of each payload is assigned. During `seed`, historical timestamps are computed values, backdated to fall within the 180-day seeded history, since that data is never intended to pass through the live ingest path. During `replay`, by contrast, each payload's timestamp is set to `time.Now()` at the exact moment the message is published to Mosquitto, not read from a pre-computed schedule. This is what makes the ingest lag metric reported for Sub-RQ 1 a measurement of genuine, live system behaviour — the real delay a transmitted reading incurs on its way through the broker and into the ingest API, as delimited in Section 7.2 — rather than a difference between two values computed ahead of time by the generator itself, which would trivially always be constant and would measure nothing about the ingest path under test. `replay` is run identically, using the same harness and configuration, against each of the three prototypes at each scale level, so that any difference in observed ingest lag or throughput reflects the architecture under test and not a difference in how the workload was generated or driven.

6.3 P1: Single-Store Baseline

P1 is the architecturally minimal configuration among the three prototypes: the ingest API writes directly to `telemetry_raw`, and all five benchmark query types (Section 5.2) are answered by MongoDB aggregation pipelines executed against this collection at request time. No background worker process and no `telemetry_aggregated` collection exist in this configuration; every read, from the point lookup of Q1 to the 180-day rollup of Q5, is computed from raw documents at the moment the query is issued.

The three compound indexes defined in Section 5.1 (Index A on `sensor.serial` and `timestamp`, Index B on `zone.zoneId` and `timestamp`, Index C on `cultivation.cultivationId` and `timestamp`) are created once, during collection initialisation, before the data generator inserts any documents. Provisioning the indexes ahead of data loading rather than after it reflects how a production deployment would establish its schema, and avoids the substantially higher cost of building compound indexes against an already-populated collection at the larger scale levels — a cost that would not represent index maintenance overhead under normal operation, but rather a one-off migration cost irrelevant to the research questions.

The Docker Compose configuration for P1 deploys only two services, Mosquitto and MongoDB, corresponding to its role as the single-store comparator against which P2 and P3 are evaluated. Because it has no background worker and no message broker dependency, P1 is also the configuration used to validate the shared ingest path in isolation, before the additional moving parts introduced by P2 and P3 are layered on top of it; this verification procedure is described in Section 6.6.

6.4 P2: CQRS Variant

P2 extends the shared ingest path with a Kafka broker, deployed in KRaft mode without a Zookeeper dependency, and a continuously running Go background worker that materialises the read model consumed by benchmark queries. In this configuration, the ingest API writes each constructed document to `telemetry_raw` exactly as in P1, and then additionally publishes it onto the `telemetry-raw-events` topic. The ordering is deliberate: the raw store is the source of truth, so if the Kafka publish fails the reading is nonetheless durably recorded and the read model can be rebuilt from it, whereas the reverse failure would lose the reading entirely. The topic is provisioned with a single partition. Given the single-node, single-consumer deployment used throughout this benchmark, one partition is sufficient to sustain the ingest rates tested at every scale level, and it has the additional benefit of preserving per-sensor event ordering: Kafka guarantees ordering only within a partition, and because every event for a given sensor is published under the same partition key (`sensor.serial`), no event for that sensor can ever be processed by the downstream worker out of order relative to another event from the same sensor.

The background worker consumes continuously from this topic and writes only to `telemetry_aggregated`; it never writes `telemetry_raw`, which the ingest API has already populated. P2 therefore retains a complete unmodified raw history exactly as P1 does, and the worker's sole responsibility is to derive the read model from the event stream. Its principal write is an upsert, performed via `replaceOne` keyed on `sensor.serial`, into the `latest_by_sensor` view inside `telemetry_aggregated`. This second write is what allows P2 to answer Q1 as a single-document lookup against a pre-materialised view, rather than as an indexed query against the full raw collection as in P1; this is precisely the architectural distinction that Sub-RQ 2 tests. The same worker also incrementally maintains the `zone_daily`, `cultivation_daily`, and `cultivation_180d_rollup` document types used to serve Q3 through Q5, updating the

relevant bucket for each event as it arrives rather than recomputing it from scratch.

Both the ingest API and the background worker in this configuration depend on a reachable Kafka broker at startup: the ingest API cannot publish events and the worker cannot consume them until the broker is available. In the current deployment this dependency is expressed directly in the Docker Compose configuration through an explicit `depends_on` relationship, so that both services wait for Kafka to become ready rather than failing outright. This dependency is, in itself, a small but real piece of evidence for the operational-complexity comparison addressed in Section 7.6: P1 has no equivalent startup ordering constraint, since it has no message broker to wait on.

6.5 P3: Lambda Architecture Variant

P3 shares its ingest path and Kafka topic with P2, but replaces the single continuous materialisation worker with two independently scheduled processing layers, corresponding to the batch and speed layers of the Lambda model introduced in Section 2.4.

The batch layer is a Go worker that runs on a fixed hourly schedule and, on each run, recomputes the `zone_daily`, `cultivation_daily`, and `cultivation_180d_rollup` aggregates from the complete raw history stored in `telemetry_raw`, using MongoDB aggregation pipelines equivalent in structure to those used by P1 at query time, but executed on a schedule rather than per request. Consistent with the immutability principle central to Lambda Architecture, each run replaces the relevant aggregate documents in full rather than updating them incrementally; this means that if a previous run produced an incorrect or partial result — due to a worker crash, a missed event, or any other transient fault — the next scheduled run overwrites it completely using the unmodified raw data, without requiring any manual correction or compensating write.

The speed layer is a Redis instance that maintains only the single most recent reading for each sensor, keyed by serial number, updated as events arrive from Kafka. Per the read-source split defined in Section 5.1, Redis is the sole source used to answer Q1 in P3: `telemetry_aggregated` in this prototype never maintains a per-sensor latest-reading view, unlike in P2, because that responsibility belongs exclusively to the speed layer. The batch layer's scope is correspondingly limited to Q3 through Q5, the queries that operate over historical windows measured in days rather than over the most recent reading. This division keeps the two layers' responsibilities disjoint by construction, rather than merely by convention, which is what allows Sub-RQ 3 to attribute any measured performance effect to the batch/speed split specifically, rather than to an incidental overlap between the two layers. Where a query would in principle require output from both layers, the merge happens at the API level rather than inside either processing layer, consistent with the serving-layer role described in Section 2.4; in the query set used by this benchmark, however, each of the five query types is answered entirely by one layer or the other, so this merge step does not arise in practice for any of Q1–Q5.

The Docker Compose configuration for P3 therefore deploys five services — Mosquitto, MongoDB, Kafka, the scheduled batch worker, and Redis — the largest component count of the three prototypes. This is one of the concrete inputs to the operational-complexity comparison in Section 7.6, alongside the two distinct codebases (continuous worker versus scheduled batch worker plus cache) that P3 requires compared to P2's single continuous worker.

6.6 Verification Prior to Full-Scale Runs

Given the cost, in both time and disk space, of running the full benchmark at the 50,000-sensor scale level, each component of the pipeline was verified incrementally at smaller scale before being trusted at full scale. This verification proceeded in three stages. First, the pure document-construction function described in Section 6.1 was unit-tested in isolation against fixed input/output pairs, confirming that a given MQTT payload and registry lookup result always produce the expected `telemetry_raw` document shape, independently of any running database or broker. Second, an end-to-end integration test exercises the full P1 ingest path against a live but minimal environment: it seeds a small `device_registry`, publishes a known MQTT payload, and asserts that the resulting document in `telemetry_raw` matches the expected enriched shape, thereby verifying that the gateway layer, the Mosquitto broker, and the document-construction logic function correctly together, not merely in isolation. Third, the data generator and ingest path were exercised together at the smallest scale level (1,000 sensors), where both throughput and ingest lag could be directly measured and inspected before committing to a run at a scale where re-running after a fault would be considerably more expensive; it was this run that produced the throughput and ingest lag figures reported in Section 6.2. Only once each of these three stages passed at the smallest scale level was the same, unmodified pipeline judged ready to be run unmodified at the 10,000- and 50,000-sensor scale levels, on the basis that the architecture and code path exercised at every scale level are identical — the scale-level parameter changes only the volume of data processed, not the logic applied to it.

7 EVALUATION

This section presents the experimental evaluation of the three prototypes described in Section 6 (P1: single-store baseline, P2: CQRS, P3: Lambda Architecture). It corresponds to the treatment validation phase of the design science engineering cycle [14]: identical benchmark workloads are applied to all three prototypes at increasing sensor scales, and the extent to which each architectural pattern improves on the single-store baseline is assessed against the metrics defined in Section 7.2.

7.1 Experimental Setup

The technology stack shared across all three prototypes (MongoDB 7.0, Mosquitto, Kafka in KRaft mode, Redis) is described in Section 6; this section covers only the run configuration used to produce the results reported below.

Hardware and deployment. All benchmark runs were executed on a single host (Apple MacBook Air M5, 512 GB SSD), with all database, broker, and worker components running as Docker containers on that host and the benchmark harness running outside Docker to avoid measurement interference. Running all components on a single host is a deliberate methodological choice consistent with controlled, reproducible benchmarking of architectural patterns

in a Docker-based environment [13]; its implications for external validity are discussed in Section 7.7.

Scale levels. The benchmark is run at three sensor scales: 1,000, 10,000, and 50,000 sensors, corresponding to approximately 17.3, 173, and 865 million documents in `telemetry_raw` (Section 5.3). A fourth scale of 100,000 sensors was planned but dropped after a disk feasibility analysis showed it would require approximately 439 GB of storage, exceeding what is available on the benchmark host alongside the operating system and the volumes of the other prototypes.

Query benchmark protocol. Each prototype is benchmarked at each scale by a single execution of the query harness. For each of the five query types, the harness performs 10 untimed warm-up iterations followed by 100 timed iterations, so every reported latency distribution is drawn from $n = 100$ samples. Target entities are drawn from `device_registry` — sensor serials for Q1 and Q2, zone identifiers for Q3, cultivation identifiers for Q4 and Q5 — and shuffled by a pseudo-random generator with a fixed seed, so that each timed iteration queries a *distinct* entity and the sequence is reproducible across prototypes. At every scale the number of available entities exceeds the 110 iterations required, so no target is reused within a query type.

This design has a consequence for how the reported distributions should be read. Because each iteration targets a different entity, the spread of the 100 samples reflects both system-level variability and genuine heterogeneity between target entities: cultivations contain between one and five sensors and are at different points in their simulated growing season, so the volume of data a given aggregation must traverse varies between iterations. The percentiles reported below are therefore percentiles over the population of entities a user might query, which is the quantity of practical interest, rather than repeated measurements of one fixed query.

Query time windows. Q2 through Q5 are defined relative to a fixed reference point rather than to wall-clock time or to the latest timestamp present in the database. The harness takes this reference point as a parameter and it is set to the same value used by the data generator to place the seeded history (2026-06-01T00:00:00Z), so that every query window falls inside the dense synthetic history at every scale and the same window is evaluated for all three prototypes.

Ingest measurement protocol. Ingest measurements are taken from a live replay in which the data generator publishes MQTT payloads at the transmission rate implied by the sensor count (Section 5.3). After the replay completes and the pipeline has drained, the harness selects all documents whose `ingestedAt` falls within an observation window ending at the latest `ingestedAt` in the collection, and computes the ingest lag of each. The observation window is set to the replay duration plus a margin, so that it captures the whole replay. Replay durations were not held uniform across prototypes and scales; the resulting per-cell observation window and sample count are therefore reported explicitly in Table 5 rather than assumed constant.

7.2 Evaluation Metrics

This subsection defines the four metric categories collected in the benchmark, explains what each measures and what it excludes, and maps each to the sub-research question it addresses. Table 3 summarises this mapping: a checkmark indicates that the metric is used to evaluate the corresponding sub-question, while a dash indicates that the metric is either not applicable or, in the case of ingest lag for Sub-RQ 2 and 3, invariant by construction and therefore not informative as a comparison.

Table 3: Evaluation metrics per sub-research question. A checkmark indicates the metric is evaluated for that question; a dash indicates it is not applicable or invariant by construction.

Sub-RQ	Write load	Query latency	Ingest lag	Oper. complexity
1 (P1 baseline profile)	✓	✓	✓	–
2 (P2 vs. P1)	✓	✓	–	–
3 (P3 vs. P1)	✓	✓	–	–
4 (operational complexity)	–	–	–	✓

Sustained write load (Sub-RQ 1). The replay is rate-limited: the data generator publishes payloads at the transmission rate implied by the sensor count, so the offered write rate is fixed by the scale level rather than being increased until the system fails to keep up. The metric collected is consequently whether each architecture sustains the offered load without accumulating a backlog, verified by comparing the number of documents written during the replay against the number the offered rate implies. This is a sustained-load check and not a capacity measurement: no prototype is driven to saturation, and the study therefore makes no claim about maximum achievable ingest rate.

Ingest lag (Sub-RQ 1). The time elapsed between the `timestamp` (device-generated event time) and `ingestedAt` (API-generated processing time) fields in each `telemetry_raw` document. The scope of this metric is determined by where `ingestedAt` is stamped: the ingest API records it at the moment it finishes constructing the enriched document, which is *before* the MongoDB insert and, in P2 and P3, before the Kafka publish and the Redis write. The measured interval therefore comprises MQTT transit, message parsing, and the `device_registry` lookup — the segment of the pipeline that is identical in all three prototypes — and excludes the database write and all architecture-specific downstream writes. The metric is accordingly used to characterise the shared ingest front-end under Sub-RQ 1 and not to compare the write-side cost of the three architectures; the consequences of this scoping decision are set out in Section 7.7.

Because the measured segment is byte-for-byte identical in all three prototypes, ingest lag is invariant with respect to the architectural variable under test by construction. It is reported for P2 and P3 in Section 7.5 as a check on that invariance rather than as a comparison, which is why Table 3 marks it as not applicable for Sub-RQ 2 and 3.

Query latency (Sub-RQ 2, Sub-RQ 3). Wall-clock time to execute each of the five benchmark query types. Each query type is executed 100 times per prototype per scale level, preceded by 10 untimed warm-up iterations, and each timed iteration targets a distinct entity (sensor, zone, or cultivation) drawn in a reproducible pseudo-random order. Latency is reported as the mean

together with the 95th percentile (p95) and 99th percentile (p99). The 95th percentile is the value below which 95% of all measured latencies fall; it captures the experience of the slowest 5% of queries and serves as a standard proxy for worst-case user-visible latency. The 99th percentile extends this to the slowest 1%, revealing the extreme tail of the distribution. The mean represents typical performance, while p95 and p99 expose whether an architecture's latency distribution is tightly concentrated or has a heavy right tail — a distinction that is often more consequential for user-facing systems than the mean alone.

Operational complexity (Sub-RQ 4). No standardised metric for operational complexity exists in the literature, so this operationalisation is defined as part of the study's contribution. It comprises four dimensions: the number of distinct infrastructure components deployed, the volume of deployment configuration required to declare them (counted in Docker Compose service definition lines), the volume of prototype-specific application code required to maintain the read model, and the number of independent write paths whose contents must be kept semantically consistent with one another. The first, second, and fourth dimensions are language-independent; the third is not, and is reported with that qualification.

Repetitions and reporting. Each configuration (prototype × scale level) is executed once. Within that execution, query latency is sampled over 100 timed iterations per query type, and ingest lag over every document written during the replay window, so each reported distribution rests on a large within-run sample. Results are reported as means with p95 and p99 values. Between-run variance is consequently not quantified; this limitation and its bearing on the conclusions are addressed in Section 7.7.

7.3 Results: Sustained Write Load

The replay is rate-limited: each sensor transmits once every 15 minutes, so the offered write rate is fixed by the sensor count at approximately 1.1, 11.1, and 55.6 writes per second for the 1,000-, 10,000-, and 50,000-sensor scales respectively. At every scale and for every prototype the number of documents ingested during the replay is consistent with the offered rate sustained for the replay duration, to within the precision of the observation window (Table 5); no prototype accumulated a backlog that persisted past the end of the replay, and pipeline drain was confirmed by polling the `telemetry_raw` document count to stability before measurement.

The correct reading of this result is that all three architectures sustain the workload the benchmark offers them, not that they have equal write capacity. Because the offered rate is capped at the transmission rate implied by the scale, no prototype was driven to saturation, and the benchmark therefore does not establish a maximum sustainable ingest rate for any of them. Write load is reported here as a precondition for the latency results that follow — all measurements were taken on systems that were keeping up with their input — and the absence of a saturation experiment is recorded as a limitation in Section 7.7.

7.4 Results: Query Latency

Table 4 reports the mean, p95, and p99 latency of each query type at each scale for all three prototypes. Each row in the table represents one combination of query type and scale level; the three column groups present the latency distribution for P1, P2, and P3 under that combination, so that performance can be compared across architectures for the same query at the same scale by reading horizontally, or across scales for the same architecture by reading vertically. Five observations follow from the table.

Q2 behaves as a control. Q2 is served from `telemetry_raw` by an identical indexed range scan in all three prototypes, and its latencies are correspondingly close: every mean lies between 1.49 and 3.28 ms, with no prototype consistently ahead of the others across scales. Neither P2 nor P3 is faster than the baseline on Q2 at any scale, which is the expected outcome and establishes that the differences observed on Q3–Q5 are attributable to pre-computation of the read model rather than to incidental infrastructure differences between the deployments.

Baseline cost scales with aggregation depth. For the baseline, latency grows with the number of raw documents an aggregation must traverse, and the growth is ordered exactly as the query windows are. Q3, aggregating one zone over 7 days, rises modestly from a mean of 1.35 ms at 1,000 sensors to 3.41 ms at 50,000. Q4, aggregating one cultivation over 30 days, rises from 5.97 to 12.96 ms. Q5, which traverses a cultivation's entire simulated history of up to 180 days, rises from 48.25 to 82.36 ms. The ordering Q3 < Q4 < Q5 holds at every scale and is a direct consequence of read-time aggregation: the baseline pays for every document within the window at the moment the query is issued. To put these numbers in practical terms: latencies below approximately 1 ms are imperceptible to a user; 1–10 ms is responsive for a dashboard widget; 50–100 ms is noticeable; and above approximately 200 ms a query begins to feel sluggish in a real-time monitoring interface. The baseline's Q3 and Q4 remain in the responsive range even at 50,000 sensors, but Q5's mean of 82 ms has already crossed into the noticeable band, and its tail — discussed below — extends well into the sluggish range.

The separated architectures are flat in scale. P2 and P3 answer Q3, Q4, and Q5 from `telemetry_aggregated` and consequently show almost no growth in the window length or in the underlying data volume. Across the whole scale range P2's Q5 mean moves only from 1.26 to 3.23 ms and P3's from 1.31 to 3.53 ms, against the baseline's 48.25 to 82.36 ms. The gap is present even at the smallest scale — Q5 at 1,000 sensors: P1 48.25 ms versus P2 1.26 ms, a factor of 38× — and widens as scale increases. At 50,000 sensors the separation reaches 25.5× on Q5 (P1 82.36 ms versus P2 3.23 ms), 14.6× on Q4 (P1 12.96 ms versus P2 0.89 ms), and 5.9× on Q3 (P1 3.41 ms versus P2 0.58 ms). The practical consequence is that P2 and P3 remain in the sub-5 ms responsive range on every query at every scale, whereas the baseline's deepest aggregation (Q5) sits squarely in the noticeable-delay range by 50,000 sensors. The residual growth that P2 and P3 do exhibit is not growth in the volume of raw telemetry but in the number of aggregate documents, which scales with the count of zones and cultivations rather than with the count of readings beneath them.

P3 leads on Q1, marginally. Q1 is the only query on which the two separated architectures differ structurally: P2 reads a single document from `telemetry_aggregated` by primary key, whereas P3 reads the value from Redis and parses it. P3 is the fastest prototype on Q1 at every scale, and its 0.22 ms mean with a 0.26 ms

Table 4: Query latency in milliseconds, by query type and scale ($n = 100$ timed iterations per cell).

		P1 (single-store)			P2 (CQRS)			P3 (Lambda)		
Query	Scale	mean	p95	p99	mean	p95	p99	mean	p95	p99
Q1 — latest reading	1K	0.24	0.30	0.37	0.19	0.23	0.29	0.14	0.17	0.18
	10K	0.97	1.42	3.51	0.51	0.74	0.98	0.25	0.72	1.11
	50K	0.88	1.13	1.35	0.76	1.39	6.44	0.22	0.25	0.26
Q2 — 24 h range scan	1K	1.49	1.69	1.99	1.90	2.31	2.51	1.74	2.13	2.26
	10K	2.49	3.11	3.24	3.28	3.69	3.74	2.05	2.51	2.77
	50K	2.42	3.22	3.66	2.29	2.60	2.92	2.77	3.19	3.27
Q3 — zone, 7 d	1K	1.35	2.83	4.26	0.29	0.53	0.59	0.27	0.51	0.60
	10K	3.10	6.98	12.23	0.88	1.07	1.25	0.67	1.29	2.55
	50K	3.41	6.50	11.61	0.58	0.71	0.81	0.81	0.93	1.08
Q4 — cultivation, 30 d	1K	5.97	12.13	21.68	0.38	0.47	0.53	0.37	0.42	0.50
	10K	12.11	25.03	39.18	1.47	1.85	1.90	0.78	0.99	1.45
	50K	12.96	27.08	42.77	0.89	1.15	1.44	1.07	1.28	1.42
Q5 — cultivation, 180 d	1K	48.25	95.73	157.92	1.26	1.42	1.45	1.31	1.45	1.60
	10K	64.61	142.44	164.99	3.24	3.88	4.18	2.59	3.03	4.10
	50K	82.36	256.05	262.23	3.23	4.40	4.95	3.53	4.35	4.66

99th percentile at 50,000 sensors confirms that the speed layer functions end to end at the largest scale tested. The margin over P2 is nevertheless a fraction of a millisecond in absolute terms.

Tail behaviour separates the architectures more sharply than the mean. The most pronounced difference is not in central tendency but in dispersion. For the baseline on Q5 the 95th percentile grows from 95.73 ms at 1,000 sensors to 256.05 ms at 50,000, and the ratio of the 95th percentile to the mean widens from approximately 2.0 to approximately 3.1 over the same range; the 99th percentile reaches 262.23 ms. In practical terms, a p95 of 256 ms means that roughly 1 in 20 full-season summary requests on a dashboard would take over a quarter of a second — perceptible delay even in isolation, and a figure that would compound under the concurrent read/write load this benchmark did not impose. On the same query P2 and P3 stay below 5 ms at every scale in both percentiles, and their 95th percentiles sit no more than 36% above their means. A baseline mean that remains below 100 ms therefore understates the user-facing cost of read-time aggregation, because the distribution over target entities is right-skewed and becomes more so as scale increases.

One counter-example deserves note. P2's Q1 at 50,000 sensors has a mean of 0.76 ms but a 99th percentile of 6.44 ms — a heavier tail than the baseline's 1.35 ms p99 on the same query. This is the only cell in the entire table where P2 has a worse tail than P1. P2 serves Q1 from a `latest_by_sensor` document set containing one entry per sensor; at 50,000 sensors this is the largest in-memory working set any prototype maintains in the benchmark. The tail is consistent with occasional MongoDB cache misses on this working set rather than with an aggregation cost: the effect does not appear at the smaller scales (where the working set fits comfortably in cache) and does not recur on any other query type.

7.5 Results: Ingest Lag

Table 5 reports ingest lag for each prototype at each scale. Each row corresponds to one prototype–scale combination and includes the observation window (in seconds) over which measurements were collected, the sample count N (the number of documents ingested during that window), and the resulting lag distribution reported as mean, p95, and p99. Observation windows and sample counts differ across cells because replay durations were not held uniform (Section 7.1); these values are reported explicitly so that the reader can assess the precision of each cell.

Table 5: Ingest lag in milliseconds. *Window* is the observation window in seconds and N the number of documents it captured.

		Observation		Ingest lag (ms)		
Prototype	Scale	Window	N	mean	p95	p99
P1 (single-store)	1K	296	329	2.83	4.6	5.7
	10K	299	3,327	1.20	3.0	4.0
	50K	295	16,388	0.66	2.0	2.0
P2 (CQRS)	1K	270	300	4.01	14.1	18.0
	10K	951	10,563	1.73	3.0	10.0
	50K	261	14,523	0.50	1.0	2.0
P3 (Lambda)	1K	958	1,065	2.55	4.0	4.4
	10K	914	10,161	1.28	3.0	3.0
	50K	936	51,984	0.65	1.0	2.0

Two observations follow. First, the three prototypes are statistically indistinguishable at every scale: at 50,000 sensors the means are 0.66, 0.50, and 0.65 ms and the 99th percentiles are 2 ms for all three. This is the expected result given where the metric is instrumented (Section 7.2) and it confirms that the shared ingest front-end is unaffected by the choice of downstream architecture. It does *not* establish that CQRS and the Lambda Architecture impose no write-side cost, because the Kafka publish that P2 and P3 perform and the Redis write that P3 performs both occur after the measurement point and are therefore outside the measured interval.

Second, mean lag *decreases* as scale increases, from between 2.5 and 4.0 ms at 1,000 sensors to below 1 ms at 50,000 for all three prototypes. This is counterintuitive — one might expect more sensors to produce more delay — but the explanation lies in the message-processing runtime rather than in the data volume. The measured interval is dominated not by data volume, which does not enter it, but by the latency of dispatching each MQTT message to the

subscriber callback. At 1.1 messages per second (1,000 sensors) the Node.js event loop is idle between messages and each one pays a cold-dispatch cost; at 55.6 messages per second (50,000 sensors) messages arrive while the loop is already active and are handled with less per-message overhead. Two properties of the measurement support this reading rather than any architectural explanation: the trend is identical in all three prototypes, including the baseline, which has no Kafka or Redis in its path at all; and the effect disappears in the tails, where the maxima across all nine cells lie between 10 and 63 ms with no monotonic relationship to scale.

Lag values are recorded at millisecond resolution, which is coarse relative to the quantity being measured at the larger scales: 46%, 56%, and 41% of the samples at 50,000 sensors for P1, P2, and P3 respectively are recorded as 0 ms. These are sub-millisecond intervals rounded down, not missing measurements, but they mean the 50,000-sensor means should be read as upper bounds on the order of magnitude rather than as precise values.

7.6 Operational Complexity

Unlike the preceding metrics, operational complexity does not follow from measured performance, and no standardised metric for it exists in the literature. It is operationalised here along the four dimensions defined in Section 7.2, each reported in Table 6: the number of distinct infrastructure components deployed, the volume of deployment configuration required to declare them, the volume of prototype-specific application code needed to maintain the read model, and the number of independent write paths whose contents must be kept consistent.

The components listed in the upper portion of the table are those that must be independently deployed, configured, and monitored as separate runtime processes; each represents a distinct failure domain that an operator must manage. The ingest API is excluded from the component count because it is shared identically across all three prototypes and therefore cancels out of any cross-prototype comparison. The benchmark harness and data generator are likewise excluded because they are test instrumentation rather than production infrastructure. Docker itself is shared deployment infrastructure common to all configurations, not an architectural component.

The upper portion of the table shows which components each prototype deploys (a checkmark indicates the component is present); the lower portion quantifies the four complexity dimensions. Reading across the columns, the ordering P1 < P2 < P3 holds on every dimension except code volume, where P2 and P3 are near-identical.

Component count orders the prototypes strictly. The baseline deploys two components and requires no prototype-specific application code beyond the shared ingest API: its read queries run aggregation pipelines directly against `telemetry_raw`. P2 adds Kafka and a continuous worker, reaching four. P3 adds Kafka, a scheduled batch worker, and Redis, reaching five. Deployment configuration volume follows the same ordering at 63, 110, and 135 lines. This dimension is language-independent and is the most robust basis for the ordering.

Code volume does not separate the two separated architectures. Excluding the shared ingest API and the one-shot routines used only to bootstrap read models from seeded history before a benchmark run, P2 requires 394 lines of Go for its continuous materialisation worker and P3 requires 384 lines — 306 of Go for the batch rewriter and 78 of JavaScript for the speed-layer write path in the ingest API. These figures are near-identical, and the small difference is not meaningful: the two counts span different languages of differing verbosity, so they are not directly comparable. Code volume therefore does not distinguish CQRS from the Lambda Architecture in this implementation, and the complexity ordering between P2 and P3 rests on the component count and on the structural argument that follows, not on this dimension.

Table 6: Operational complexity by prototype. Configuration lines are Docker Compose service declarations; application code counts only prototype-specific code, excluding the ingest API logic common to all three prototypes and the one-shot routines used to bootstrap read models before a benchmark run.

	P1	P2	P3
Mosquitto (MQTT broker)	✓	✓	✓
MongoDB	✓	✓	✓
Apache Kafka (KRaft)	–	✓	✓†
Redis (speed layer)	–	–	✓
Background worker	–	continuous	hourly batch
Infrastructure components	2	4	5
Compose configuration (lines)	63	110	135
Prototype-specific code (lines)	0	394	384
Write paths to keep consistent	1	2	3

† Deployed and written to, but not consumed; see below.

The number of write paths that must be kept consistent. The dimension that does separate P2 from P3 is how many independent paths write state that queries later read, each of which must be kept semantically consistent with the others. The baseline has one: the ingest API writes `telemetry_raw` and queries read it. P2 has two: the ingest API writes the raw store and the continuous worker maintains the aggregate collection from the same event stream. P3 has three: the ingest API writes the raw store, the batch worker periodically recomputes the aggregate collection from it, and the ingest API separately writes the Redis speed layer, whose contents must agree with what the batch layer will eventually produce. Reconciling a batch layer and a speed layer that compute the same quantity by different means is the recurring maintenance cost that motivated later criticism of the pattern [9], and it is present here in the same form.

Kafka is deployed in P3 but not consumed. One asymmetry in the P3 realisation should be stated explicitly. Its batch worker does not consume from Kafka; it reads `telemetry_raw` directly through aggregation pipelines, and the speed layer is written synchronously by the ingest API rather than fed from the stream. Kafka is therefore deployed, configured, and published to on every ingested message, but nothing reads it in the measured P3 configuration. This departs from the canonical formulation, in which the incoming stream feeds both layers [10], and it means P3 carries the full operational and configuration cost of a message broker, plus a synchronous publish on every write, for no function that this benchmark exercises.

7.7 Threats to Validity

Ingest lag instrumentation does not span the full write path. The most consequential threat concerns the scope of the ingest lag metric. Because `ingestedAt` is stamped before the MongoDB insert and before the Kafka and Redis writes, the metric captures only the shared front-end of the pipeline (Section 7.2). The architecture-specific write costs — a synchronous Kafka publish in P2 and P3, and an additional synchronous Redis write in P3 — are real, are performed inline in the same request handler, and are entirely outside the measured interval. The write side of Sub-RQ 1 must therefore be read narrowly: the benchmark shows that the shared ingest front-end is unaffected by the downstream architecture, not that the downstream architecture is free.

No saturation experiment. The replay is rate-limited to the transmission rate implied by each scale, so no prototype was driven to its capacity limit (Section 7.3). The write-side results establish that each architecture sustains the offered load, and nothing about maximum throughput. Since P2 and P3 perform strictly more synchronous work per ingested message than the baseline, a saturation experiment is where a write-side penalty for architectural separation would be expected to appear.

Single run per configuration. Each prototype-scale combination was benchmarked once, with the reported distributions drawn from the 100 timed iterations within that run rather than from repeated independent runs. Between-run variance — from container scheduling, page cache state, and host background activity — is therefore not quantified. The within-run sample is large and the effects reported on Q3–Q5 span one to two orders of magnitude, well beyond any plausible run-to-run variation, but the smaller differences reported on Q1 and Q2 should not be treated as established.

Non-uniform ingest observation windows. Replay durations and observation windows differ across the cells of Table 5, so sample counts differ by up to a factor of three at the same scale. Since ingest lag is a per-message quantity measured in a steady state whose offered rate is fixed by the scale, a longer window yields more samples of the same distribution rather than a different distribution, and the comparison between prototypes remains valid. The differing N nevertheless means the cells are not equally precise, and the protocol should be made uniform in any replication.

Single-host deployment. All components run as containers on one physical machine rather than across a distributed cluster. This reflects the scale at which precision agriculture IoT platforms of the size tested are realistically deployed, but it removes inter-component network latency from every measurement and limits generalisation to genuinely distributed, multi-node production deployments.

Synthetic workload and absence of concurrent load. The benchmark uses a synthetic dataset rather than production telemetry. Synthetic-but-realistic workloads are an accepted substitute for production data in architectural benchmarking when production data is unavailable or cannot be shared [7]. Separately, query latency and ingest measurements were taken in isolation from one another, with no concurrent read and write traffic; the implications of this are examined in Section 8.3.

The Lambda pattern rather than the Lambda stack. P3 implements its batch layer as a Go worker running periodic MongoDB aggregation pipelines rather than with a distributed processing framework such as Apache Spark. The architectural distinction under test — periodic batch recomputation combined with a speed-layer overlay, as against continuous incremental materialisation — is preserved, but the absolute batch-side figures are specific to this realisation.

Scale ceiling and workload framing. The 100,000-sensor scale was excluded for disk feasibility reasons (Section 7.1), not because it was expected to be uninformative, and findings at 50,000 sensors should not be extrapolated beyond that scale. At 1,000 to 50,000 sensors on a single Docker host the workload also does not meet canonical big data definitions on volume or velocity; the architectural benefits observed should be framed as relevant to small-to-mid-scale precision agriculture deployments rather than as evidence about big data workloads generally.

8 DISCUSSION

This section interprets the results reported in Section 7, drawing out their implications for each sub-research question, weighing the query-latency gains of the separated architectures against the operational complexity they introduce, and setting out the limitations that bound these conclusions and the future work they motivate.

8.1 Interpreting the Results by Sub-Research Question

Sub-RQ 1: characterising the single-store baseline. The baseline with standard compound indexes serves point lookups and short-range scans (Q1, Q2) with sub-3 ms mean latency at every scale, including 50,000 sensors, and its ingest path sustains the full offered write rate at every scale, with a mean lag across the shared ingest front-end that never exceeds 3 ms and falls to 0.66 ms at the largest scale. Its weakness is confined to queries that aggregate over deep time windows. For the 30-day trend (Q4) the mean rises from 5.97 ms at 1,000 sensors to 12.96 ms at 50,000, and for the 180-day summary (Q5) from 48.25 ms to 82.36 ms. The more consequential signal is in the tail rather than the mean: Q5's 95th percentile grows from 95.73 ms to 256.05 ms, and its ratio of p95 to mean widens from about 2.0 to about 3.1 across the range. A mean that remains sub-100 ms understates the user-facing risk, because it is the tail that determines how often a dashboard request feels slow, and the tail is what deteriorates fastest. The baseline is thus not so much slow as *increasingly variable* under read-time aggregation, and that variability is the quantity the separated architectures are designed to remove.

Sub-RQ 2: CQRS against the baseline. CQRS improves on the baseline for every query answered from the pre-computed read model (Q1, Q3, Q4, Q5) at every scale, and the improvement is largest exactly where the baseline is weakest. For Q5, P2 holds a mean of 1.26–3.24 ms across the whole scale range while the baseline climbs to 82.36 ms, a separation that reaches roughly 25× at 50,000 sensors; for Q4 the corresponding factor is roughly 15×. Crucially, P2's latency on these queries is essentially scale-invariant, because the cost of reading a materialised aggregate depends on the number of zone- and cultivation-level documents returned rather

than on the volume of raw telemetry beneath them. The one query on which P2 does not improve, Q2, is the control: it reads `telemetry_raw` directly in all three prototypes, and its near-identical latencies confirm that the Q3–Q5 gains come from pre-computation of the read model and not from incidental infrastructure differences. On the write side the answer is narrower. P2 sustained the offered load at every scale with an ingest lag indistinguishable from the baseline's, so CQRS imposes no write-side penalty this benchmark can detect — but because the load was rate-limited and the instrumentation covers only the shared ingest front-end (Section 7.7), that is a statement about the absence of detectable cost rather than evidence of equal write capacity. Sub-RQ 2 is therefore answered unambiguously on latency and provisionally on throughput: for aggregation-heavy access patterns, CQRS converts a volume-dependent, tail-heavy latency profile into a flat one, at no write-side cost observable under the conditions tested.

Sub-RQ 3: Lambda against the baseline. P3 reproduces P2's read-side behaviour almost exactly, which is expected, since the two serve Q3–Q5 from the same aggregate collection and differ only in whether that collection is maintained by continuous incremental updates (P2) or by periodic batch recomputation with a speed-layer overlay (P3). Where P3 distinguishes itself is Q1, the latest-reading lookup, which it serves from the in-memory Redis speed layer: it is the fastest prototype on Q1 at every scale (0.14 ms at 1,000 sensors, 0.22 ms at 50,000), and the successful Q1 responses at the largest scale confirm the speed layer functions end to end there. In absolute terms, however, this advantage is small — a fraction of a millisecond over P2— and it does not extend to the aggregation queries, on which P3 and P2 are effectively tied. Its write-side result mirrors P2's: the offered load was sustained at every scale and ingest lag was indistinguishable from the baseline's, subject to the same caveat about rate-limited load and front-end-only instrumentation. P3 does, however, perform the most synchronous work per ingested message of the three — a Kafka publish and a Redis write in addition to the raw insert — so it is the prototype for which the unmeasured write-side cost is largest in principle. Lambda thus answers Sub-RQ 3 affirmatively but adds little that CQRS does not already provide on the read side.

Sub-RQ 4: operational complexity. The complexity ordering is strict and increases monotonically, but not along every dimension measured. P1 runs two components, needs 63 lines of deployment configuration, and requires no prototype-specific application code beyond the shared ingest API. P2 runs four components and 110 configuration lines; P3 runs five and 135. Code volume, by contrast, does not separate the two: P2's continuous worker is 394 lines and P3's batch rewriter and speed-layer write path together are 384, and the difference between those figures is neither large nor meaningful, since they span two languages of differing verbosity (Section 8.3). What does separate them is structural. P1 has a single write path whose output queries read; P2 has two, the raw store and the worker-maintained aggregate collection; P3 has three, because its speed layer is written independently of the batch layer and the two must agree on quantities they compute by different means. P3 compounds this with an unconsumed message broker: it publishes every ingested message to Kafka, and pays the configuration, deployment, and per-message cost of doing so, while its batch worker reads the raw collection directly. Sub-RQ 4 therefore records P3's additional operational surface as real and attributable to component count and write-path multiplicity rather than to implementation effort — the price paid for capabilities that, in this workload, the read-side results did not show to be necessary.

8.2 The Performance–Complexity Trade-off

Taken together, the four answers turn the thesis's central question — not which architecture is fastest, but when the speed is worth the complexity — into a decision that depends on scale.

At 1,000 sensors the baseline's absolute latencies are already modest: even Q5, its worst case, completes in a mean of 48 ms, well within budget for a dashboard that refreshes on the order of seconds. Introducing CQRS or Lambda at this scale at least doubles the component count for a benefit the end user is unlikely to perceive. Here the simplicity of the baseline is a feature, and the evidence does not support paying for architectural separation.

At 50,000 sensors the balance shifts. The baseline's Q5 tail of 256 ms, though still sub-second in isolation, signals the growing variability that would compound under the concurrent read/write load of a production system — a condition this benchmark did not impose and which would fall hardest on the baseline, whose aggregation reads and ingest writes contend for the same collection and indexes. P2 removes that variability entirely (Q5 p95 of 4.40 ms) for the cost of two extra components, a doubled deployment configuration, and a single additional write path to keep consistent. P3 matches P2's read performance but adds a fifth component and a third write path without reducing the amount of application code that must be maintained, and its one distinctive advantage — the sub-millisecond Redis lookup on Q1 — is marginal against that additional surface.

The trade-off therefore resolves, for this workload class and these scales, in favour of CQRS. It captures the full read-latency benefit of separating reads from writes with one component fewer, one write path fewer, and no unconsumed message broker, while the Lambda Architecture's batch-plus-speed-layer design delivers no measurable aggregation-query advantage and requires reconciling two derivation paths for the same quantities — precisely the recurring cost that motivated later criticism of the pattern [9]. The single-store baseline remains a defensible choice at the lower end of the range (1,000–10,000 sensors), where its simplicity outweighs a latency penalty that is real but not yet operationally significant.

8.3 Limitations

Beyond the threats to validity enumerated in Section 7.7, three limitations qualify the interpretation above and deserve separate emphasis.

Code volume is a weak proxy for complexity. The operational-complexity comparison includes counts of prototype-specific code, but that code is not written in a single language: the aggregation workers are implemented in Go while the speed-layer write path lives in the JavaScript ingest API, and P3's figure combines both. Line counts are not comparable across languages of differing verbosity, and in this case they do not discriminate at all — P2 and P3 come out within ten lines of one another. The honest conclusion is that the Lambda Architecture does not cost

more *code* than CQRS here; it costs more *components* and more independent write paths. The complexity ordering therefore rests on those two dimensions, which are language-independent, and the line counts are reported for transparency rather than as evidence for the ordering.

The write side is only partly observed. The conclusion that architectural separation is obtained without a write-side penalty is weaker than the ingest-lag figures make it appear. Because `ingestedAt` is stamped before the database insert and before the Kafka and Redis writes, the measured interval covers only the ingest front-end that all three prototypes share (Section 7.7), and the replay never drives any prototype to saturation. P2 and P3 do perform strictly more synchronous work per ingested message than P1, and that work is invisible to both metrics. What the results establish is that this additional work is not large enough to create a backlog at the offered rates tested; whether it narrows the headroom at rates closer to capacity is an open question, and one that bears directly on the recommendation below, since it is the only dimension on which the separated architectures might prove worse rather than better.

Absence of concurrent load. Each query type was measured in isolation, with no competing read or write traffic. In production, ingest and dashboard queries contend for the same MongoDB resources, and that contention falls most heavily on the baseline, whose read-time aggregations and write path share a collection and its indexes; the pre-computed read models of P2 and P3 decouple these I/O paths by construction. The latency gap reported here may therefore *understate* the real-world advantage of architectural separation under load, which means the recommendation in favour of separation at larger scales is, if anything, conservative.

The Lambda pattern rather than the Lambda stack. Because P3 implements its batch layer as a Go worker rather than with Apache Spark (Section 7.7), this study evaluates the Lambda *pattern* — batch recomputation combined with a speed-layer cache — rather than the Lambda *stack* as originally specified [10]. A distributed Spark batch layer might behave differently at scales beyond those tested, particularly where the batch recomputation itself becomes the bottleneck. The architectural distinction under test is preserved regardless of batch-layer technology, but the absolute batch-side figures are specific to the Go-and-MongoDB realisation and should not be read as characterising a Spark-based deployment.

8.4 Future Work

Several extensions follow directly from these limitations. Introducing concurrent read and write load would test the hypothesis, advanced above, that the baseline degrades faster under contention and that the measured gap therefore understates the benefit of separation. Two changes to the write-side experiment would close the gap identified above: instrumenting the ingest path so that the timestamp is taken after the database write and after the Kafka and Redis writes, which would make the per-message cost of architectural separation directly measurable; and replacing the rate-limited replay with a saturation experiment that raises the offered rate until each prototype fails to keep up, which would establish maximum sustainable ingest rate and reveal whether the extra synchronous work in P2 and P3 reduces write capacity. Extending the scale range beyond 50,000 sensors, on hardware with sufficient storage, would reveal whether the baseline's Q5 tail continues to grow linearly or turns non-linear. Substituting an Apache Spark batch layer for the Go worker would allow the canonical Lambda stack to be measured directly and its batch-side cost compared against CQRS's continuous worker. A multi-node deployment, in which inter-component network latency is no longer negligible, would test whether the single-host findings generalise to distributed production environments. Finally, evaluating the Kappa Architecture [9] — which collapses the batch and speed layers into a single stream-processing path and was proposed specifically to remove the dual-codebase cost identified here — would complete the architectural comparison space and directly address the principal operational objection to the Lambda Architecture surfaced by this study.

9 CONCLUSION

This thesis set out to establish empirically when the separation of read and write concerns is worth its operational cost for precision agriculture IoT telemetry. Three prototype data architectures — a single-store MongoDB baseline, a CQRS variant materialising a read model from a Kafka event stream, and a Lambda Architecture variant combining periodic batch recomputation with a Redis speed layer — were implemented over a common ingest path, storage engine, and synthetic dataset, and benchmarked at 1,000, 10,000, and 50,000 sensors on five query types spanning point lookups to full-season aggregations.

The results answer the four sub-questions with a consistent shape. The single-store baseline is not slow in general: it serves point lookups and short-range scans in under 3 ms at every scale, and its shared ingest path keeps up with the offered write rate throughout. Its weakness is confined to read-time aggregation, where latency grows with the depth of the time window — from a mean of 48 to 82 ms on the 180-day summary across the scale range — and where the tail deteriorates faster than the mean, the 95th percentile reaching 256 ms at 50,000 sensors. Both separated architectures remove that growth almost entirely, holding the same query below 5 ms at every scale, a separation of roughly 25× at the largest scale tested. They do so for the same reason and to almost exactly the same degree: the cost of reading a materialised aggregate depends on the number of zones and cultivations, not on the volume of raw telemetry beneath them. The Lambda Architecture's one distinctive gain, a sub-millisecond latest-reading lookup served from its speed layer, is a fraction of a millisecond over the CQRS equivalent, and it does not extend to any aggregation query.

Operational complexity separates the two more sharply than performance does. Component count, deployment configuration volume, and the number of independent write paths that must be kept semantically consistent all order the three prototypes strictly, while the volume of application code does not distinguish CQRS from Lambda at all. The Lambda prototype's additional cost is thus structural rather than a matter of implementation effort: a fifth component, a third write path, and the obligation to reconcile a batch layer and a speed layer that compute the same quantities by different means.

For the workload class and scale range studied, the trade-off resolves in favour of CQRS. It captures the full read-latency benefit of separating reads from writes at a lower operational complexity

than the Lambda Architecture, whose batch-plus-speed-layer design delivered no measurable advantage on aggregation queries in these experiments. The single-store baseline remains a defensible choice at the lower end of the range, where its latency penalty is real but not yet operationally significant and its simplicity is a genuine asset.

These conclusions are bounded by the conditions under which they were obtained. All measurements come from a single-host, containerised deployment driven by a synthetic workload at a rate-limited offered load, with query and write measurements taken in isolation from one another, and the ingest instrumentation observes only the portion of the write path that the three architectures share. The architectural comparison they support is therefore about read-side behaviour under aggregation-heavy access patterns, which is where the difference between these patterns is decided for this class of platform, and about the operational surface each pattern requires to obtain it. Whether the separated architectures also pay a write-side price at rates closer to capacity remains open, and is the most direct extension of this work.

REFERENCES

[1] G. Andreadis, L. Versluis, F. Mastenbroek, and A. Iosup. A reference architecture for datacenter scheduling: Design, validation, and experiments. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC '18)*, pages 37:1–37:15, Dallas, TX, USA, 2018. IEEE/ACM.

[2] A. Avritzer, V. Ferme, A. Janes, B. Russo, A. van Hoorn, H. Schulz, D. Menasché, and V. Rufino. Scalability assessment of microservice architecture deployment configurations: A domain-based approach leveraging operational profiles and load tests. *Journal of Systems and Software*, 165:110564, 2020.

[3] G. Blinowski, A. Ojdowska, and A. Przybyłek. Monolithic vs. microservice architecture: A performance and scalability evaluation. *IEEE Access*, 10:20357–20374, 2022.

[4] M. U. Demirezen and T. S. Navruz. Performance analysis of lambda architecture-based big-data systems on air/ground surveillance application with ADS-B data. *Sensors*, 23(17):7580, 2023.

[5] M. M. Eyada, W. Saber, M. M. El Genidy, and F. Amer. Performance evaluation of IoT data management using MongoDB versus MySQL databases in different cloud environments. *IEEE Access*, 8:110656–110668, 2020.

[6] M. Fowler. CQRS. https://martinfowler.com/bliki/CQRS.html, 2011. Accessed: June 2026.

[7] G. Hesse. *A Benchmark for Enterprise Stream Processing Architectures*. PhD thesis, Universität Potsdam, 2022.

[8] A. Kamilaris, A. Kartakoullis, and F. X. Prenafeta-Boldú. A review on the practice of big data analysis in agriculture. *Computers and Electronics in Agriculture*, 143:23–37, 2017.

[9] J. Kreps. Questioning the Lambda architecture. O'Reilly Radar, https://www.oreilly.com/radar/questioning-the-lambda-architecture/, 2014. Accessed: June 2026.

[10] N. Marz and J. Warren. *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning, 2015.

[11] M. Overeem, M. Spoor, S. Jansen, and S. Brinkkemper. An empirical characterization of event sourced systems and their schema evolution — lessons from industry. *Journal of Systems and Software*, 178, 2021.

[12] D. Van Landuyt, J. Benaouda, V. Reniers, A. Rafique, and W. Joosen. A comparative performance evaluation of multi-model NoSQL databases and polyglot persistence. In *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*, pages 286–293. ACM, 2023.

[13] D. Vinken. Comparative performance analysis of distributed web applications running in three common virtualization scenarios. MSc thesis, University of Amsterdam, 2022.

[14] R. J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Springer, 2014.

[15] S. Wolfert, L. Ge, C. Verdouw, and M. J. Bogaardt. Big data in smart farming — a review. *Agricultural Systems*, 153:69–80, 2017.

[16] G. Young. CQRS documents. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf, 2010. Accessed: June 2026.

[17] M. A. Zamora-Izquierdo, J. Santa, J. A. Martínez, V. Martínez, and A. F. Skarmeta. Smart farming IoT platform based on edge and cloud computing. *Biosystems Engineering*, 177:4–17, 2019.

[18] X. Zhang, Z. Cao, and W. Dong. Overview of edge computing in the agricultural Internet of Things: Key technologies, applications, challenges. *IEEE Access*, 8:141748–141761, 2020.

A ETHICAL CONSIDERATIONS

This study uses synthetic data generated to simulate agricultural IoT sensor telemetry. In the benchmark, no real sensor data, personal data, or production system data is used. The study does not involve human subjects. The benchmark prototypes run in isolated Docker containers on development hardware and do not interact with any production infrastructure.

B AI USAGE

In the interest of transparency, and following the Vrije Universiteit's guidelines on the use of generative AI in student work, this section records how AI assistance was used in producing this thesis. Large language models (primarily Anthropic's Claude) were used substantially, in three distinct roles.

Implementation, debugging, and automation. AI assistance was used extensively throughout the construction of the three prototypes, the data generator, the benchmark harness, and the aggregation workers: writing and refactoring code, diagnosing defects, and producing the shell scripts that automate seeding, replay, measurement, and teardown for each prototype and scale level. This was the largest single use. Several substantive defects in the benchmark were identified through this collaboration, including the query-window anchoring error and the uninitialised read models described in Section 7.1, both of which had silently invalidated an earlier round of measurements. All generated code was reviewed, executed, and validated against expected behaviour before being relied upon, and every reported measurement was produced by code in the public repository rather than by any model output.

Literature search and orientation. Because this thesis addresses a combination of topics — CQRS and the Lambda Architecture applied specifically to agricultural IoT telemetry — for which little directly comparable prior work exists, AI assistance was also used extensively as a research aid: to identify candidate literature, to suggest adjacent fields worth surveying, and to summarise unfamiliar material as an entry point. This use was treated as a search and orientation tool only. Every work cited in this thesis was retrieved independently, read in the original, and verified for authorship, venue, year, and identifier before citation. No citation in the bibliography rests on a model's description of a source, and no claim attributed to a source was accepted without confirming it in the source itself. This precaution was taken deliberately, given the known tendency of language models to produce plausible but non-existent references.

Drafting and editing of the written thesis. AI assistance was used in drafting and revising portions of the written text, including structuring sections, producing prose from outlines, notes, and measured results, and tightening wording. All such text was reviewed, checked against the underlying data, and revised by the author, who takes full responsibility for the accuracy of every claim, number, and interpretation presented. The research questions, the architectural designs under test, the benchmark methodology, and the conclusions drawn from the results are the author's own.

C ARTIFACT DESCRIPTION

The complete artifact — the three prototype configurations, the data generator, the benchmark harness, the aggregation workers, the automation scripts, and the raw result files reported in Section 7 — is publicly available at:

https://github.com/Aleboy2/CS-Bachelor-Thesis

Repository layout. Each prototype is defined in its own Docker Compose file (`docker-compose.p1.yml`, `.p2.yml`, `.p3.yml`), with a separate named Docker volume per prototype so that data from one prototype's run cannot leak into another's. This is a reproducibility requirement as well as a fairness one, since a stale volume from a previous run would silently invalidate the scale-level document counts reported in Section 5.3. The `mongo-init/` scripts define the collections and indexes required before either the data generator or the ingest API writes its first document, and are shared unmodified across all three prototype configurations. The `datagen/`, `harness/`, and `worker/` directories contain the Go data generator, benchmark harness, and aggregation workers respectively; `ingest-api/` contains the shared Node.js ingest service; and `results/` contains the measurement CSV files from which every figure in Section 7 is computed.

Prerequisites. A Docker installation, a Go toolchain to build the `datagen` and `harness` binaries (both run on the host, outside any container), and free disk space of approximately 3.4 GB per prototype at the 1,000-sensor scale, 34 GB at 10,000, and 172 GB at 50,000. At the two larger scales the prototypes must be seeded, measured, and torn down one at a time; holding all three 50,000-sensor volumes simultaneously requires roughly 516 GB and will exhaust a 512 GB host. Docker's own disk allocation must be raised accordingly before seeding at 50,000 sensors.

Reproduction procedure. Figure 6 gives the full command sequence for one prototype-scale combination. Substitute `p1`, `p2`, or `p3` and the target sensor count throughout, and repeat for each combination, tearing the stack down with `-v` between combinations so that the next run begins from an empty volume. The complete, checkpointed 50,000-sensor pipeline is also available as `run-benchmark-50k.sh`, which runs all three prototypes in sequence and resumes from the last completed step if interrupted; seeding at that scale takes several hours per prototype, so resumability matters in practice.

Correctness checks that must not be skipped. Three steps in the sequence above exist because omitting them produces results that appear valid but measure nothing, and each corresponds to a defect that occurred during this study.

First, the seeded document count must be verified (step 4) rather than assumed. A pruned or partially seeded volume yields fast queries over an almost empty collection.

Second, the read models of P2 and P3 must be bootstrapped from the seeded history (step 5) and not left to be populated by live replay traffic alone. A read model containing only the last few minutes of replayed data returns empty results for the historical aggregation queries; because these are cursor-based reads, an empty result is not an error, so the harness records a fast, successful, and entirely meaningless measurement.

Third, the query harness must be anchored to the same reference date used when seeding (step 7). Anchored instead to wall-clock time, the Q2–Q5 windows fall past the end of the synthetic history into an empty interval, with the same silent consequence.

```
# 1. Build the host binaries (run once)
cd datagen && go build -o datagen . && cd ../harness && go build -o harness . && cd ..

# 2. Start the target prototype (substitute p1 | p2 | p3)
docker compose -f docker-compose.p2.yml up -d

# 3. Seed 180 days of history. --benchmark-start fixes the reference date that
#    the query harness must later be anchored to.
./datagen/datagen seed --sensors 10000 --days 180 \
    --benchmark-start "2026-06-01T00:00:00Z" --seed 42 --drop

# 4. Verify the seed actually landed before continuing.
mongosh "mongodb://localhost:27017/iot_benchmark" \
    --eval 'db.telemetry_raw.countDocuments({})'

# 5. Bootstrap the read model from the seeded history. P1 skips this step.
#    P2:
docker compose -f docker-compose.p2.yml run --rm worker --mode p2-backfill
#    P3: confirm the batch worker's startup rewrite finished, then seed Redis.
#    The keys carry a 3600 s TTL, so run step 7 within 60 minutes of this.
docker compose -f docker-compose.p3.yml run --rm worker --mode p3-redis-backfill

# 6. Replay live traffic, then wait for the pipeline to drain (poll the
#    telemetry_raw count until it is stable across three consecutive checks).
./datagen/datagen replay --sensors 10000 --duration 5m --seed 42

# 7. Measure. The --anchor value must match --benchmark-start from step 3.
./harness/harness ingest-lag --window 6m \
    --output results/ingest-lag-p2-10k.csv
./harness/harness query --prototype p2 --anchor "2026-06-01T00:00:00Z" \
    --iterations 100 --warmup 10 --seed 42 \
    --output results/queries-p2-10k.csv

# 8. Verify 100 rows per query type, then tear down and free the volume.
awk -F, 'NR>1 {n[$2]++} END {for (q in n) print q, n[q]}' results/queries-p2-10k.csv
docker compose -f docker-compose.p2.yml down -v
```

Figure 6: Full command sequence for one prototype-scale combination.